

Hessian-basierte Vorhersage von Quantisierungsschäden in CNNs

Schichtweise Sensitivitätsanalyse unter Power-of-Two Quantisierung

Dominic Jean-Luc Tarnowski

Betreuung: Prof. Thomas Manke

Hochschule für Technik und Wirtschaft Berlin

12. September 2026

Zusammenfassung

Dieser Bericht dokumentiert das Forschungsprojekt B. Im Zentrum steht die Frage, ob sich mithilfe Hessian-basierter Krümmungsmetriken vorhersagen lässt, welche Schichten eines Convolutional Neural Network unter der Power-of-Two-Quantisierung (PoT) am stärksten geschädigt werden, wie Post-Training-Quantisierung und Quantization-Aware-Training diese Krümmung dabei jeweils unterschiedlich prägen, und in welchem Ausmaß der Schaden überhaupt auf Gewichts- statt Aktivierungsquantisierung zurückgeht. Die PoT ist ein hardware-attraktives Quantisierungsschema, da sie Multiplikationen durch Bitshifts ersetzt. Gleichzeitig erzeugt sie jedoch ungleichmäßige Genauigkeitsverluste über die Schichten hinweg. Als etablierter Ansatz schlägt die HAWQ-V2-Formulierung hierfür das Produkt aus Hessian-Spur und Störungsgröße, $\text{Tr}(H) \cdot \|\Delta W\|^2$, als Sensitivitäts-Score vor. Um den Schaden jeder Schicht isoliert zu bestimmen, wird jeweils nur diese eine Schicht quantisiert und der daraus resultierende Anstieg des Validierungs-Loss gemessen. Auf dieser Grundlage werden drei Scores gegen den gemessenen Schaden getestet, die rohe Krümmung, die Störungsgröße und das HAWQ-V2-Produkt, und zwar über drei Architekturen (CNN, ResNet-18, ResNet-50) sowie zwei Datensätze (CIFAR10, ImageNet100). Die Ergebnisse zeigen, dass kein Score universell zuverlässig ist, und dass auch die rohe Krümmung nur ein eng umrissenes, schwaches Signal liefert: Sie identifiziert die am stärksten geschädigte Schicht (Rang 1) in 3 von 6 Architektur×Datensatz-Kombinationen, doch schließen 5 von 6 Bootstrap-Konfidenzintervallen der Rangkorrelation Null ein; im einzigen signifikanten Fall (ResNet-50/CIFAR10) platziert dieselbe Krümmung die wahre Top-Schicht auf Rang 40 von 54. Zwei der drei Rang-1-Erfolge erweisen sich zudem als direkt an `conv1` gebunden, das selbst überwiegend ein Fusions-/Fan-in-Artefakt ist, und verschwinden, sobald `conv1` aus der Schichtmenge entfernt wird. Zwei günstige Baselines ganz ohne Hessian-Berechnung, Gradientennorm und empirische Fisher-Diagonale, erreichen bei einem Bruchteil des Rechenaufwands dieselbe oder eine leicht bessere Rang- und top- k^* -Trefferquote wie die rohe Krümmung. Das HAWQ-V2-Produkt wird zusätzlich durch eine Anti-Korrelation zwischen Störungsgröße und tatsächlichem Schaden in die Irre geführt. Zur Ursachenanalyse der Krümmungsspitzen werden eine Random-Init-Kontrolle, eine gescheiterte KFAC-Zerlegung, eine Dekomposition in Gewichts- und Aktivierungsanteile sowie eine Deployment-Evaluation auf realer Hardware angewandt. Dabei zeigt sich, dass die PoT-Quantisierung auf einer NVIDIA A100 keinen Laufzeitvorteil bringt, da aktuelle INT8-Kernel für Faltungsschichten fehlen, während ein CPU-Pfad mit echten INT8-Kernen (fbgemm, CIFAR10) einen realen Durchsatzgewinn misst, allerdings für generische INT8-Arithmetik und nicht für den PoT-Bitshift selbst, der auf keiner der beiden Plattformen nativ verfügbar und somit ungemessen bleibt. Ein Rekonziliations-Ledger, das dokumentiert, wie stark Trace-Werte von Konfigurationsentscheidungen abhängen, die in der Literatur meist unausgesprochen bleiben, stellt ein weiteres Ergebnis dar.

1 Einleitung

1.1 Motivation und Problemstellung

Neuronale Netze werden zunehmend auf ressourcenbeschränkter Hardware ausgeführt, etwa auf eingebetteten Systemen, mobilen Geräten und Edge-Beschleunigern. Auf solcher Hardware sind die Inferenzkosten eines vollpräzisen Modells oft nicht mehr

tragbar. Die Quantisierung reduziert diese Kosten, indem die Gewichte und Aktivierungen eines Netzes von 32-Bit-Gleitkommazahlen (FP32) auf niedriger aufgelöste, diskrete Werte abgebildet werden, typischerweise auf 8 Bit oder weniger.

Eine besonders hardware-attraktive Variante ist die Power-of-Two-Quantisierung (PoT). Hierbei werden die Gewichte auf ein Gitter der Zweierpotenzen abgebildet. Der Vorteil liegt in der Arithmetik,

da eine Multiplikation mit einer Zweierpotenz einem einfachen Bitshift entspricht. Dadurch können die sonst dominierenden Multiplizierer-Schaltkreise in der Hardware entfallen [21, 24]. Dieser Effizienzgewinn ist jedoch mit einem Nachteil verbunden. Da das PoT-Gitter logarithmisch und nicht linear ist, variiert der Quantisierungsfehler stark und unvorhersehbar mit den tatsächlichen Gewichtswerten einer Schicht.

Um zu entscheiden, welche Schichten eines Netzes eine niedrigere Bit-Breite vertragen und welche nicht, hat sich die Verwendung von Krümmungsinformation aus der zweiten Ableitung des Verlustes etabliert, also der Hessian-Matrix. Ein Ansatzes ist die HAWQ-Familie [4, 3]. HAWQ rangiert die Schichten nach ihrem größten Hessian-Eigenwert, während HAWQ-V2 dies zu einem Produkt aus Hessian-Spur und quadrierter Störungsgröße, $\text{Tr}(H) \cdot \|\Delta W\|^2$, verfeinert, motiviert durch eine Taylor-Entwicklung des Verlustes zweiter Ordnung. Beide Arbeiten nutzen diese Formulierung, um eine Bit-Zuteilung über das gesamte Netz zu optimieren. Validiert wird dabei die End-to-End-Genauigkeit nach der Zuteilung und nicht die Vorhersagegüte des Scores auf Ebene einzelner Schichten.

1.2 Forschungsfragen

Das Projekt verfolgt drei Forschungsfragen:

1. Wie prägen PTQ und QAT die schichtweise Verlust-Krümmung (gemessen über die Hessian-Spur) von CNNs unter PoT-Quantisierung jeweils unterschiedlich?
2. Sagt die schichtweise Verlust-Krümmung (Hessian-Spur) vorher, welche Schichten tatsächlich den größten PoT-Gewichts-Quantisierungsschaden erleiden, und wie haengen diese beiden Auswahlen (vorhergesagt vs. tatsächlich) von Architektur und Datensatz ab?
3. In welchem Ausmaß ist der PoT-induzierte Genauigkeitsverlust auf Gewichts- gegenüber Aktivierungsquantisierung zurückführbar, und für welches Regime ist demzufolge die Hessian-basierte Sensitivitätsanalyse anwendbar?

2 Grundlagen

2.1 Quantisierung neuronaler Netze

Ein trainiertes neuronales Netz speichert seine Gewichte und berechnet seine Aktivierungen typischer-

weise als 32-Bit-Gleitkommazahlen (FP32). Quantisierung bildet diese kontinuierlichen Werte auf eine kleinere, diskrete Menge von Repräsentanten ab, üblicherweise mit weniger Bits pro Wert. Das reduziert den Speicherbedarf direkt proportional zur Bit-Breite und kann, wenn die Zielhardware quantisierte Arithmetik-Einheiten besitzt, auch die Rechenzeit senken [6, 23].

Es gibt zwei grundsätzliche Strategien, ein Netz zu quantisieren. Bei der Post-Training-Quantisierung (PTQ) wird ein bereits fertig trainiertes FP32-Modell nachträglich quantisiert, ohne weiteres Training. PTQ ist schnell und benötigt keine Trainingsdaten in großem Umfang, produziert aber typischerweise größere Genauigkeitsverluste, da das Modell nie gelernt hat, mit dem Quantisierungsfehler umzugehen [23]. Beim Quantization-Aware-Training (QAT) wird der Quantisierungseffekt bereits während des Trainings simuliert, sodass das Modell seine Gewichte an die Quantisierung anpassen kann. QAT erzielt in der Regel höhere Genauigkeit, benötigt aber zusätzliches Training [6].

Sowohl bei der PTQ-Kalibrierung als auch bei QAT wird der Quantisierungseffekt üblicherweise durch eine Fake-Quantization simuliert [15]. Dabei wird ein Wert quantisiert und sofort wieder in Gleitkomma zurückkonvertiert (oft als „QDQ“ bezeichnet), sodass der Rest der Berechnung weiterhin in FP32 abläuft, den Quantisierungsfehler aber dennoch „sieht“ [15]. Das eigentliche Problem liegt in der Rundungsfunktion in der Mitte dieser Operation, deren Ableitung fast überall null ist und die den Gradientenfluss beim Backpropagation-Training blockieren würde. Der Straight-Through-Estimator (STE, [1]) löst dies, indem er im Vorwärtsdurchlauf die tatsächliche, nicht-differenzierbare Quantisierung anwendet, im Rückwärtsdurchlauf jedoch so verfährt, als wäre die Quantisierungsfunktion die Identität. Der Gradient wird also unverändert durchgereicht. Diese Relaxierung ist für dieses Projekt relevant, da sie auch bestimmt, welche Krümmung auf quantisierten Modellen tatsächlich gemessen wird (§2.2).

Während die uniforme Quantisierung die Werte auf ein gleichmäßig verteiltes Gitter abbildet (etwa 256 Stufen bei 8 Bit), rundet die PoT-Quantisierung jeden Gewichtswert auf die nächste Zweierpotenz innerhalb eines Exponentenbereichs [21]. Der Vorteil ist arithmetischer Natur, da eine Multiplikation $x \cdot 2^k$ auf Digitalhardware einem Bitshift um k Stellen entspricht. Dieser ist billiger als eine allgemeine Multiplikation und ohne dedizierten Multiplizierer realisierbar [24]. Der Nachteil ist ein nicht-uniformes Feh-

lverhalten. Da das PoT-Gitter logarithmisch ist, ist die Gitterauflösung nahe null sehr fein und wird zu größeren Beträgen hin exponentiell groeher. Ob eine Schicht durch die PoT-Rundung stark oder kaum gestört wird, hängt somit weniger von einer festen Bit-Breite ab, wie bei der uniformen Quantisierung, als davon, wie gut die Verteilung der Gewichtswerte einer Schicht zufällig mit dem PoT-Gitter übereinstimmt [21].

Für die Reproduzierbarkeit von $\|\Delta W\|^2$ (§2.3) ist das konkrete Gitter maßgeblich. Die in diesem Projekt verwendete Implementierung (`src/quantization/quantizer.py`) rundet elementweise, ohne Skalierungsfaktor und ohne Zero-Point. Für einen Gewichtswert w gilt

$$e(w) = \text{clamp}(\text{round}(\log_2 |w|), e_{\min}, e_{\max}), \quad Q(w) = \text{sign}(w) \cdot 2^{e(w)}$$

mit dem festen Exponentenbereich $e_{\min} = -8$, $e_{\max} = 1$. Dieser Bereich ist ein fester Wert des Codes und nicht datenabhängig kalibriert; er gilt unverändert für jede Schicht, jedes Modell und jeden Datensatz. Das erreichbare Gitter ist damit $Q_{\text{PoT}} = \{\pm 2^e : e \in \{-8, -7, \dots, 1\}\}$, insgesamt 20 Werte. Der Fall $w = 0$ wird intern durch $w \rightarrow 10^{-9}$ ersetzt, um $\log_2(0)$ zu vermeiden. Da $\text{sign}(10^{-9}) = +1$, quantisiert ein Gewicht von 0 auf $+2^{-8}$ statt auf 0. Der Wert 0 liegt damit nicht im tatsächlich erreichbaren Wertebereich von Q_{PoT} .

2.2 Die Hessian-Matrix und Krümmungsanalyse

Für eine Verlustfunktion $L(W)$, die von den Gewichten W eines Netzes abhängt, ist die Hessian-Matrix $H = \nabla_W^2 L$ die Matrix der zweiten partiellen Ableitungen des Loss nach jedem Gewichtspaar. Sie beschreibt, wie stark sich die Steigung des Loss ändert, wenn man sich entlang einer Richtung im Gewichtsraum bewegt, anschaulich gesprochen also, wie stark die Verlustlandschaft an der aktuellen Position gekrümmt ist.

Quantisierung stört die Gewichte um einen kleinen Betrag $\Delta W = W - Q(W)$, wobei $Q(\cdot)$ die Quantisierungsfunktion ist. Eine Taylor-Entwicklung des Loss um den unquantisierten Punkt zweiter Ordnung ergibt

$$\Delta L \approx \nabla L^\top \Delta W + \frac{1}{2} \Delta W^\top H \Delta W, \quad (1)$$

wobei der lineare Term bei einem konvergierten Modell (Gradient nahe null) vernachlässigbar ist, sodass der quadratische, Hessian-gewichtete Term den Loss-Anstieg dominiert [3]. Diese Beziehung ist die theoretische Grundlage dafür, die Krümmung als Proxy

für die Quantisierungsempfindlichkeit zu verwenden. Laut Gleichung (1) sollte eine Schicht mit hoher Krümmung empfindlicher auf eine Störung gleicher Größe reagieren als eine Schicht mit niedriger Krümmung.

Für moderne Netze mit Millionen Parametern ist die vollständige Hessian-Matrix weder speicherbar noch berechenbar. Praktikabel ist jedoch die Spur $\text{Tr}(H) = \sum_i H_{ii}$, die die aufsummierte Krümmung über alle Gewichtsrichtungen angibt. Der Hutchinson-Schätzer approximiert die Spur stochastisch. Für zufällige Vektoren v mit $\mathbb{E}[vv^\top] = I$ (üblicherweise Rademacher-Vektoren, deren Komponenten gleichverteilt ± 1 sind) gilt $\text{Tr}(H) = \mathbb{E}[v^\top H v]$ [29]. Das Produkt Hv , ein Hessian-Vektor-Produkt, lässt sich effizient per automatischer Differentiation berechnen, ohne H jemals explizit zu bilden. Mehrere solcher Zufallsproben werden gemittelt, um die Schätzung zu stabilisieren.

Yao u. a. [29] implementieren diesen Hutchinson-Ansatz (inklusive Top-Eigenwert-Schätzung über Power-Iteration) als wiederverwendbares Werkzeug für PyTorch-Modelle und etablieren ihn als De-facto-Standard für Krümmungsanalysen in der Quantisierungsliteratur. Dieses Projekt verwendet PyHessian als Grundlage der eigenen Trace-Messungen (§4).

2.3 HAWQ und HAWQ-V2

Dong u. a. [4] führen HAWQ ein. Dabei werden die Schichten nach ihrem größten Hessian-Eigenwert rangiert, und Schichten mit größerem Eigenwert erhalten mehr Bits in einem gemischt-präzisen Quantisierungsschema. Dong u. a. [3] verfeinern dies zu HAWQ-V2. Statt nur des Eigenwerts wird nun ein Sensitivitäts-Score als Produkt aus Hessian-Spur und quadrierter Störungsgröße gebildet, $\text{Tr}(H_l) \cdot \|\Delta W_l\|^2$ für Schicht l , direkt motiviert durch Gleichung (1), wobei $\text{Tr}(H_l)$ als Stellvertreter für die Krümmung dient.

H_l ist dabei der Diagonalblock der vollen Hessian-Matrix, der nur die Parameter von Schicht l betrifft, nicht die vollständige Hessian über alle Schichten hinweg. Kreuz-Schicht-Terme zweiter Ordnung, also wie stark sich eine Störung von Schicht l und eine Störung von Schicht m gegenseitig im Loss auswirken, gehen in $\text{Tr}(H_l)$ nicht ein. Das passt zur Zielgröße dieser Analyse, denn auch der gemessene Schaden `abs_loss_damage(l)` betrifft jeweils nur eine einzelne isoliert quantisierte Schicht (§4.3). Score und Zielgröße beschränken sich damit beide auf dieselbe Einzelschicht-Störung, und Kreuz-Schicht-Kopplung bleibt in beiden unberücksichtigt.

Beide verwenden den Score, um eine globale Bit-Zuteilung über alle Schichten zu optimieren, und validieren den Erfolg dieser Zuteilung anhand der End-to-End-Genauigkeit des gesamten Netzes nach der Quantisierung. Keine der beiden Arbeiten testet direkt, ob der Score auf Ebene einzelner Schichten tatsächlich mit dem dort auftretenden Schaden korreliert [4]. Ein Netz kann insgesamt eine gute Endgenauigkeit erreichen, obwohl der Score einzelne Schichten falsch einordnet, solange sich die Fehler kompensieren oder die insgesamt zugeteilten Bits ausreichen. Genau diese Lücke, die per-Schicht-Vorhersagegüte des HAWQ-V2-Scores direkt gegen gemessenen Schaden zu testen, adressiert dieses Projekt.

2.4 Hessian-Spektrum und Random-Matrix-Theorie

Über die Spur hinaus lässt sich das Spektrum, also die Menge der Eigenwerte, der Hessian-Matrix untersuchen. Sagun, Bottou und LeCun [25] und Sagun u. a. [26] zeigen empirisch, dass das Hessian-Spektrum tiefer neuronaler Netze typischerweise aus zwei Teilen besteht, einem Bulk nahe null, der die meisten Eigenwerte enthält, und wenigen großen Ausreißer-Eigenwerten, die mit der Anzahl der Klassen des Klassifikationsproblems zusammenhängen. Diese Bulk-Ausreißer-Trennung motiviert die Vorstellung, dass ein Netz effektiv nur entlang weniger Richtungen scharf gekrümmt ist.

Karakida, Akaho und Amari [12] liefern hierzu eine theoretische Grundlage aus der Random-Matrix-Theorie (RMT). Für zufällig initialisierte, untrainierte Netze lässt sich die Fisher-Information, die eng mit der Hessian-Matrix nahe einem Minimum verwandt ist, im Mittelfeld-Limit vorhersagen, inklusive einer charakteristischen Tiefenabhängigkeit. Diese Vorhersage ist der theoretische Bezugspunkt für die Random-Init-Kontrolle dieses Projekts (Phase 2, §5.3). Sie erlaubt zu unterscheiden, ob eine beobachtete Krümmungsspitze in einer bestimmten Schicht bereits bei Initialisierung vorhanden ist („architektonisch“, durch die Netzstruktur selbst erklärbar) oder erst durch Training entsteht („gelernt“).

Eine weitere Zerlegungsidee ist die Kroneckerfaktorierte Näherung der Krümmung (KFAC, Martens und Grosse [20]) sowie ihre Erweiterung auf Faltungsschichten (KFC, Grosse und Martens [7]). Die Grundidee besteht darin, die Fisher-Information einer einzelnen Schicht näherungsweise als Kronecker-Produkt zweier kleinerer Matrizen zu faktorisieren, $F \approx A \otimes G$, wobei A die Kovarianz der

Schicht-Eingaben und G die Kovarianz der Verlust-Gradienten bezüglich der Schicht-Ausgaben erfasst. Gelingt diese Faktorisierung, so ließe sich die Spur einer Schicht in einen Eingabe-getriebenen und einen Gradienten-getriebenen Anteil zerlegen, was eine potenziell aufschlussreiche Diagnose dafür wäre, warum eine Schicht hohe Krümmung aufweist. Sowohl die RMT-Vorhersage als auch die KFAC-Zerlegung waren für dieses Projekt vielversprechende Werkzeuge, um die architektonischen Ursachen beobachteter Krümmungsspitzen zu erklären (Phase 6, §5.8). Wie in §5 gezeigt wird, führte insbesondere die KFAC-Zerlegung im PoT-Kontext jedoch nicht zum Ziel, da die vorhergesagte und die gemessene Spur um mehrere Größenordnungen voneinander abweichen.

2.5 Aktivierungsquantisierung und Range-Ausreißer

Alle bisherigen Konzepte betreffen die Quantisierung von Gewichten. Ein separates, in der Literatur gut dokumentiertes Problem ist die Quantisierung von Aktivierungen, also der Zwischenausgaben einer Schicht während der Inferenz. Aktivierungen folgen einem anderen Schadensmechanismus als Gewichte. Statt gleichmäßig verteilter Fehler treten oft wenige, aber extreme Ausreißer-Kanäle auf, deren Wertebereich um Größenordnungen über dem Rest liegt. Ein einzelner Skalierungsfaktor pro Tensor muss diesen gesamten Bereich abdecken, was die effektive Auflösung für alle übrigen, unauffälligen Kanäle drastisch reduziert. Xiao u. a. [28] (SmoothQuant) und Lin u. a. [19] (AWQ) zeigen dies für große Sprachmodelle und schlagen vor, die Schwierigkeit vor der Quantisierung von den Aktivierungen auf die leichter quantisierbaren Gewichte zu verschieben. Li u. a. [18] (BRECQ) verfolgt einen alternativen, blockweisen Rekonstruktionsansatz für PTQ, der Aktivierungs- und Gewichtsquantisierung gemeinsam über kleine Netzblöcke optimiert.

Für dieses Projekt ist relevant, dass Aktivierungsausreißer kanalweise auftreten und primär von der Eingabeverteilung bestimmt werden, während die Hessian-Krümmung eine Eigenschaft der Gewichte relativ zum Loss ist. Es gibt keinen a-priori-Grund, warum ein Hessian-basierter Score, der für Gewichtsschaden entwickelt wurde, auch Aktivierungsschaden vorhersagen sollte. Diesen Punkt greift die Dekomposition in Gewichts- und Aktivierungsanteile (Phase 1, §4.5) empirisch auf.

3 Verwandte Arbeiten

HAWQ [4] und HAWQ-V2 [3] etablieren die Hessian-Krümmung als Signal fuer die Bit-Zuteilung in gemischt-präzisen Quantisierungsschemata (§2.3). Beide Arbeiten validieren ihren Ansatz End-to-End, das heißt, nach der Bit-Zuteilung wird die Genauigkeit des gesamten Netzes gemessen und nicht die Vorhersagegüte des Scores für einzelne Schichten. Diese Validierungsstrategie ist für den ursprünglichen Zweck, eine gute globale Zuteilung zu finden, angemessen. Sie lässt jedoch offen, ob der Score tatsächlich lokal korrekt ist, denn ein Netz kann trotz einer schlecht kalibrierten Einzelschicht-Rangfolge insgesamt gut abschneiden, wenn genügend Gesamt-Bit-Budget vorhanden ist oder sich die Fehler zwischen den Schichten teilweise kompensieren.

Hill [9] testen als erste Arbeit direkt, ob die HAWQ-V2-Metrik $\text{Tr}(H) \cdot \|\Delta W\|^2$ mit tatsächlich gemessenem Quantisierungsschaden korreliert und nicht nur mit der End-to-End-Genauigkeit nach der Bit-Zuteilung. Das Setting dieser Arbeit ist jedoch ein großes Sprachmodell (LLM) unter uniformer Quantisierung und kein Vision-CNN unter PoT. Da PoT, wie in §2.1 erläutert, ein anderes, nicht-uniformes Fehlverhalten aufweist, ist unklar, ob sich die dortigen Befunde auf PoT-CNNs übertragen lassen.

Miyashita, Lee und Murmann [21] fuehren PoT als hardware-freundliche logarithmische Gewichtsrepräsentation ein, Lee u. a. [16] zeigen den zugehörigen Energie-Vorteil logarithmischer Berechnung für CNNs, und Przewłocka-Rus u. a. [24] motivieren PoT zusätzlich für Hardware mit niedriger Bit-Breite. Neuere Arbeiten wie Kim u. a. [14] und Li u. a. [17] verfeinern die PoT-Gitterformulierung selbst, um bei sehr niedriger Bit-Breite höhere End-to-End-Genauigkeit zu erreichen, und Elgenedy [5] überträgt PoT auf Sprachmodell-Gewichte. Alle diese Arbeiten fokussieren, wie Miyashita, Lee und Murmann [21] und Przewłocka-Rus u. a. [24], auf die Machbarkeit, End-to-End-Genauigkeit und den Hardware-Vorteil von PoT und nicht auf eine schichtweise Sensitivitätsanalyse. In der PoT-Praxis, auch in diesem Projekt (§4), ist es zudem üblich, die erste Faltungsschicht (`conv1`) entweder von der Quantisierung auszunehmen oder ihr besondere Aufmerksamkeit zu widmen, da sie mit ihren wenigen Eingabekanälen (meist drei Farbkanaäle) und dem dadurch kleinen Fan-in empirisch besonders empfindlich reagiert. Dieses Projekt liefert mit der Random-Init-Kontrolle (Phase 2) und der Fusion-Analyse (Phase 3.5) eine Erklärung dafür, warum `conv1` in dieser

Hinsicht eine Sonderrolle einnimmt.

SmoothQuant [28] und AWQ [19] zeigen, dass Aktivierungsschaden primär durch kanalweise Ausreißer getrieben wird, also durch einen von der Gewichtskrümmung unabhängigen Mechanismus (§2.5). BRECCQ [18] adressiert PTQ durch eine blockweise Rekonstruktion, die Gewichts- und Aktivierungsfehler gemeinsam optimiert, ohne die Hessian-Krümmung als Sensitivitätssignal zu nutzen. Diese Arbeiten motivieren, warum dieses Projekt Gewichts- und Aktivierungsschaden trennt (Phase 1, §4.5), statt beide unter einem gemeinsamen Score zu behandeln.

Vor diesem Projekt existiert kein publizierter empirischer Test, der die HAWQ-V2-Formulierung direkt gegen gemessenen, per-Schicht-isolierten Gewichtsschaden für Vision-CNNs unter PoT prüft.

4 Methodik

4.1 Experimentelles Setup

Fuer dieses Projekt werden drei Convolutional-Neural-Networks (CNNs) untersucht. Ein kompaktes, projektspezifisches `cnn` mit sechs quantisierbaren Schichten (vier Faltungs-, zwei vollverbundene Schichten), welches sich nach dem von Shahmustafa Mujawar et. al. aus dem Jahr 2018 beschriebenen Modell orientiert [22]ue, `ResNet-18` [8] und `ResNet-50` [8]. Beide ResNet-Varianten werden ohne vortrainierte Gewichte instanziiert (`resnet18_no_weights`, `resnet50_no_weights`) und vollstaendig von Grund auf trainiert. Die drei Architekturen spannen eine Größenordnung an Schichtzahl auf ($n = 6, 21, 54$), was für die Normalisierung der Top- k -Auswertung in §4.4.1 relevant wird.

Als Datensätze werden CIFAR10 [2] (32×32 Pixel, 10 Klassen) und ImageNet100 [27, 10] (224×224 Pixel, ein 100-Klassen-Subset von ImageNet) verwendet. Mit diesen Daten haben wir sowohl kleine als auch groessere Eingaben was es uns erlaubt, Effekte, die von der Eingabeauflösung abhängen (etwa die Speicher- und Rechenkosten der Hessian-Schätzung, §4.2), von architekturbedingten Effekten zu trennen.

Jede der sechs Architektur×Datensatz-Kombinationen wird mit einem Trainings-Seed trainiert. Für jede Kombination werden das FP32-Basismodell, eine PTQ-Variante und eine QAT-Variante trainiert. Trainings-Metadaten sind in Tabelle 1 zusammengefasst.

Die PoT-Implementierung (`src/quantization/quantizer.py`) rundet jeden Gewichtswert elementweise auf die nächste Zweierpotenz innerhalb eines festen Exponentenbereichs, ohne Skalierungsfaktor und ohne Zero-Point. Derselbe Bereich gilt unverändert für jede Schicht, jedes Modell und jeden Datensatz; die genaue Definition des Gitters und die Randbehandlung bei $w = 0$ stehen in §2.1.

4.2 Kanonische Trace-Konfiguration

Gemessene Hessian-Trace-Werte haengen empfindlich von Konfigurationsentscheidungen ab, die die Literatur meist implizit lässt. Um reproduzierbare, vergleichbare Trace-Werte über die Trainingsläufe und Schichten hinweg zu erhalten, wird vor der Hauptanalyse eine kanonische Konfiguration festgelegt, dokumentiert in `trace_config.json`:

- **Reduktion:** Cross-Entropy.
- **Basis:** fusionierte Batch-Normalisierung [11] als kanonisch für stufenübergreifende Vergleiche, da PTQ- und QAT-Checkpoints ausschließlich in fusionierter Form existieren. FP32-Modelle werden zusätzlich in unfusionierter Form vermessen, sodass der Fusionseffekt selbst sichtbar bleibt.
- **Datensplit:** feste Validierungsbilder.
- **Proben-Seeds:** drei feste Seeds (42, 43, 44), deren Ergebnisse zu Mittelwert $\pm$ Standardabweichung pro Schicht aggregiert werden.
- **Determinismus:** deterministische Algorithmen, `eval()`-Modus (kein Dropout, feste BN-Statistiken).

Da ein Hessian-Vektor-Produkt bei 224×224 -Eingaben (ImageNet100) um ein Vielfaches teurer ist als bei 32×32 (CIFAR10), sowohl in der Rechenzeit als auch im Speicherbedarf, wurde das Probe-Budget an den jeweiligen Datensatz angepasst. 80 Bilder mit 100 Hutchinson-Iterationen für CIFAR10 gegenüber 24 Bildern mit 30 Iterationen für ImageNet100. Dieses ungleiche Budget ist eine dokumentierte Limitation (§6.6), denn Messrauschen könnte zur schwächeren Korrelation auf CIFAR10 beitragen, unabhängig von einem echten Datensatzeffekt.

Da PTQ- und QAT-Gewichte über den Straight-Through-Estimator (§2.1) quantisiert wurden, ist die auf diesen Modellen gemessene Hessian-Matrix genau genommen die STE-relaxierte Krümmung des

Loss, also die Krümmung, die sich ergibt, wenn die nicht-differenzierbare Quantisierungsfunktion im Rückwärtsdurchlauf durch die Identität ersetzt wird, und nicht die klassische Hessian eines unquantisierten Netzes. Diese Unterscheidung ist für die Interpretation der PTQ- und QAT-Trace-Werte in §5 wichtig.

4.3 Gewichts-Schaden-Isolation

Um den Schaden einer einzelnen Schicht l zu isolieren, werden ausschließlich die Gewichte dieser einen Schicht per PoT quantisiert, während alle übrigen Gewichte sowie alle Aktivierungen im gesamten Netz auf FP32 verbleiben. Implementiert wird dies, indem die Fake-Quantization-Module für Gewicht (`weight_fake_quant`) und Aktivierung (`act_fake_quant`) aller anderen Schichten durch `nn.Identity()` ersetzt werden.

Für jede Schicht l wird der Loss-Schaden definiert als

$$\text{loss_damage}(l) = \text{loss}_{\text{iso}}(l) - \text{loss}_{\text{fp32}}, \quad (2)$$

wobei $\text{loss}_{\text{iso}}(l)$ der mittlere Validation-CrossEntropy-Loss des Modells mit isoliert quantisierter Schicht l ist und $\text{loss}_{\text{fp32}}$ der Loss des unveränderten FP32-Modells. Ein positiver Wert bedeutet, dass der Loss durch die Quantisierung gestiegen ist. Als primäres Ranking-Ziel wird $\text{abs_loss_damage}(l) = |\text{loss_damage}(l)|$ verwendet, da unter QAT auch negative Werte auftreten (die isolierte Quantisierung einer Schicht kann den Loss gegenüber der FP32-Baseline sogar senken, siehe §4.5) und ein reines Ranking nach Betrag Schäden beider Vorzeichen gleich gewichtet.

Die Genauigkeit ist stückweise konstant und daher unempfindlich gegenüber kleinen Verschiebungen der Ausgabewahrscheinlichkeiten. Die Hessian-Matrix ist per Definition die Krümmung des Loss (§2.2), und Gleichung (1) sagt den Loss-Anstieg vorher. Ein direkter Test der HAWQ-V2-Motivation muss daher denselben Loss als Schaden-Ziel verwenden, für den die Taylor-Entwicklung überhaupt aufgestellt wurde, was dazu führt, dass Loss entsprechend als Damage-Metrik dieser Analyse gesetzt ist. Ergänzend wird geprüft, ob die Kernbefunde robust gegenüber dieser Wahl sind, indem dieselbe Analyse zusätzlich mit genauigkeitsbasiertem Schaden wiederholt wird (§5.12).

4.4 Drei Sensitivitäts-Scores

Fuer die Sensitivitäts-Analyse werden drei Scores gegen $\text{abs_loss_damage}(l)$ verglichen:

$$S_{\text{raw}}(l) = \text{Tr}(H_l), \quad (3)$$

$$S_{\text{pert}}(l) = \|W_l - Q(W_l)\|^2, \quad (4)$$

$$S_{\text{hawq}}(l) = \text{Tr}(H_l) \cdot \|W_l - Q(W_l)\|^2. \quad (5)$$

S_{raw} ist die rohe kanonische Krümmung. S_{pert} ist die quadrierte Störungsgröße, die durch PoT-Rundung entsteht. S_{hawq} ist das HAWQ-V2-Produkt [3].

S_{raw} und S_{pert} sind die zwei multiplikativen Faktoren, aus denen S_{hawq} gebildet wird, und keiner der beiden Faktoren enthält Information über den jeweils anderen. Der Vergleich aller drei Scores testet daher direkt, ob der Schaden von der Krümmung, von der Störungsgröße oder tatsächlich vom Produkt beider getrieben wird, statt gegen beliebige externe Alternativen zu testen.

Zusätzlich zu den drei Hessian-basierten Scores werden drei Baselines gegen dieselbe Zielgröße getestet, die keine Hessian-Berechnung benötigen:

$$S_{\|W\|}(l) = \|W_l\|_F, \quad (6)$$

$$S_{\nabla}(l) = \|\nabla_{W_l} \mathcal{L}\|_{\text{Konsistent}F}, \quad (7)$$

$$S_{\text{fisher}}(l) = \sum_i \|\nabla_{W_l} \mathcal{L}_i\|_F^2, \quad (8)$$

wobei $\mathcal{L}$ der über den Probe-Batch mittelwert-reduzierte Verlust ist und $\mathcal{L}_i$ der Verlust für ein einzelnes Beispiel i desselben Batches. S_{∇} ist ein Rückwärtsdurchlauf über den vollständigen Probe-Batch. S_{fisher} braucht einen echten Pro-Beispiel-Gradienten, der aus einem einzigen mittelwert-reduzierten Rückwärtsdurchlauf nicht rekonstruierbar ist, und wird deshalb mit Batch-Größe 1 pro Durchlauf berechnet. Alle drei Baselines verwenden denselben Probe-Batch und dieselbe Verlustreduktion wie S_{raw} (§4.2), damit der Vergleich fair ist.

Jeder dieser Scores wird auf drei Arten ausgewertet. Einmal die Spearman-Rangkorrelation zwischen den Score-Werten und abs_loss_damage über alle Schichten, mit 95%-Bootstrap-Konfidenzintervall ($B = 2000$ Resamples, Ziehen von Schichten mit Zurücklegen), den Rang, den der Score der Schicht mit dem tatsächlich größten Schaden zuweist (Rang 1 = perfekt) und die Überlappung zwischen den top- k^* vom Score als am schädlichsten eingestuften Schichten und den top- k^* tatsächlich am meisten geschädigten Schichten.

4.4.1 Großennormalisierung der Top- k -Auswertung

Da die drei Architekturen eine Größenordnung an Schichtzahl umspannen ($n = 6, 21, 54$), wäre ein fester Top-5-Schwellwert für das `cnn` fast das gesamte Netz, fuer ResNet-50 dagegen nur ein kleiner Ausschnitt und damit kein über die Architekturen hinweg vergleichbarer Maßstab. Verwendet wird daher $k^* = \lceil 0.1n \rceil$, sodass alle drei Architekturen und alle drei Scores am gleichen Top-Dezil-Maßstab gemessen werden.

4.5 Gewichts- vs. Aktivierungs-Dekomposition

Um die dritte Forschungsfrage zu beantworten, wird der Gesamt-Genauigkeitsverlust einer Kombination in drei Konfigurationen zerlegt. Volle Quantisierung, nur Gewichte quantisiert, nur Aktivierungen quantisiert. Für letzteres werden Beobachter zur Aktivierungs-Kalibrierung auf einem Kalibrierungs-Batch eingesetzt, wie bei Standard-PTQ üblich. Aus dem Vergleich der resultierenden Genauigkeiten werden `weight_loss_pts` und `activation_loss_pts` abgeleitet, und die `dominant_source` wird als die Konfiguration mit dem größeren Betrag bestimmt. Ergänzend werden die Aktivierungs-Wertebereiche pro Schicht sowie der darin auftretende Ausreißerfaktor analysiert.

4.6 Weitere Analysephasen

Über Phase 1 und die Kernanalyse aus §4.3–4.4 (im Ergebnisteil als „Phase 4“ geführt) hinaus umfasst die Pipeline vier weitere Phasen, deren Methodik hier kurz skizziert und deren Ergebnisse in §5 berichtet werden. Abbildung 1 zeigt die sechs Phasen im Ueberblick.

In Phase 2 wird unterschieden, ob eine beobachtete Krümmungsspitze in einer Schicht „architektonisch“ (bereits bei zufälliger Initialisierung vorhanden) oder „gelernt“ (erst durch Training entstanden) ist. Dabei wird die PyHessian-Trace auf drei unabhängig zufällig initialisierten, untrainierten Modellen pro Architektur (drei Init-Seeds) gemessen und das resultierende Trace-Profil mit dem des trainierten FP32-Modells verglichen.

In Phase 3 werden Sweeps über die in §4.2 genannten Konfigurationsknöpfe durchgeführt, da unterschiedlich konfigurierte Läufe im Projektverlauf teils stark abweichende Trace-Werte für dieselbe nominelle Größe produziert hatten.

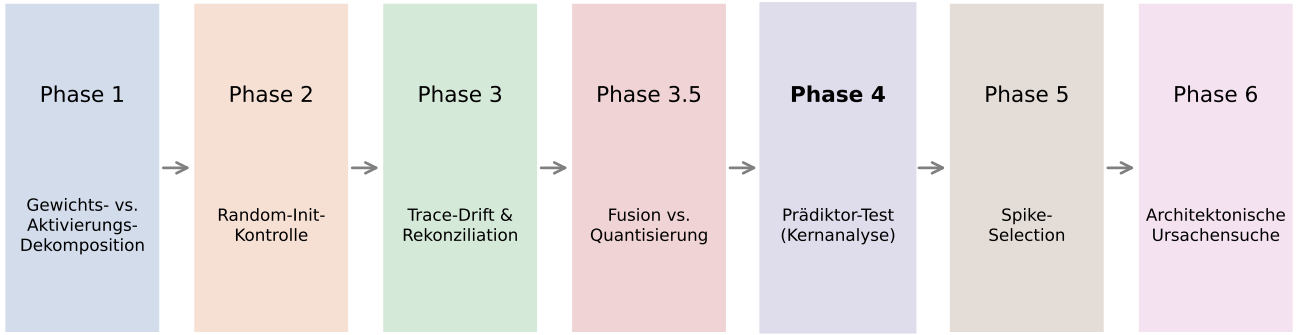


Abbildung 1: Sechs-Phasen-Analyse-Pipeline. Erzeugt von `notebooks/12_pipeline_overview.ipynb`.

In Phase 3.5 wird die Batch-Normalisierungs-Fusion betrachtet, die die effektiven Gewichte einer Schicht bereits vor jeder Quantisierung verändert. Um den Fusion-Effekt vom eigentlichen PoT-Quantisierungseffekt zu trennen, werden die Trace-Werte in unfusionierter FP32-Basis, fusionierter FP32-Basis und PTQ-Basis verglichen, und eine Schicht wird als „quant_induced“ definiert, wenn die PTQ-Trace über die bereits durch Fusion erhöhte FP32-Trace hinausgeht, andernfalls als „fp32_intrinsic“.

Phase 5 vergleicht die Schicht mit der höchsten Trace („Trace-Spike“) direkt mit der Schicht mit dem größten gemessenen Schaden pro Modell, um zu prüfen, ob die naive Heuristik „höchste Krümmung = größter Schaden“ zutrifft.

Letztes in Phase 6 wird für die jeweilige Trace-Spike-Schicht jedes Modells versucht, die beobachtete Krümmungserhöhung über die KFAC-Faktorisierung (§2.4) in einen Eingabe-getriebenen (A) und einen Gradienten-getriebenen (G) Anteil zu zerlegen und die vorhergesagte Pro-Parameter-Trace mit der tatsächlich gemessenen zu vergleichen. Ergänzend werden größen-kontrollierte Residuen berechnet (Korrelation von Trace-Ausreißern mit strukturellen Deskriptoren wie Fan-in, Fan-out und Output-Map-Große, unter Kontrolle der Parameterzahl).

4.7 Deployment-Evaluation

Um zu prüfen, ob der theoretische Bitshift-Vorteil von PoT (§2.1) auf realer Hardware tatsächlich in eine kürzere Inferenzzeit übergeht, werden zwei Deployment-Pfade evaluiert. Der erste Pfad ist `torchao` 0.17 auf einer NVI-

DIA A100-GPU (Compute Capability 8.0). Dabei werden `torchao` INT8-Konfigurationen (`Int8DynamicActivationInt8WeightConfig` für die dynamische Aktivierungsquantisierung, `IntxWeightOnlyConfig` fuer die reine Gewichtsquantisierung) auf alle sechs Kombinationen angewendet und die Durchsatz-Zeit über eine Reihe von Batch-Groößen vermessen, jeweils gegen die FP32-Baseline auf derselben GPU. Der zweite Pfad ist ein CPU-Pfad über den `fbgemm`-Backend [13] von PyTorchs eager-mode-PTQ, der auf x86-Hardware mit AVX-512-VNNI-Unterstützung (Zielsystem: AMD Ryzen 7 7700X, Zen 4) tatsächlich quantisierte Conv2d- und Linear-Kernel einsetzt, im Gegensatz zum `torchao`-GPU-Pfad.

5 Ergebnisse

5.1 Kern-Genauigkeiten

Tabelle 1 fasst die FP32-, PTQ- und QAT-Genauigkeit für alle sechs Architektur×Datensatz-Kombinationen zusammen (Quelle: `pipeline_summary.csv`, `ptq_summary.csv`, `qat_summary.csv`). PTQ verursacht einen Genauigkeitsverlust, der auf ImageNet100 größer ausfällt als auf CIFAR10 (etwa `cnn`: −17.98 vs. −10.22 Prozentpunkte). QAT übertrifft in jeder Kombination sogar die FP32-Baseline, am deutlichsten beim `cnn` auf ImageNet100 (FP32 43.04 %, QAT 51.18 %, +8.14 Punkte gegenüber FP32, +26.12 Punkte gegenüber PTQ).

Auffällig ist, dass PTQ auf CIFAR10 (kleine 32×32 -Eingaben) für die ResNets moderat ausfällt (−1.4 bis −1.6 Punkte), auf ImageNet100 aber massiv (−8.8 bis −16.0 Punkte). Diese Asymmetrie deutet

Tabelle 1: FP32-, PTQ- und QAT-Validierungsgenauigkeit (%) für alle sechs Kombinationen, mit PTQ-Verlust und QAT-Verbesserung gegenüber PTQ (Prozentpunkte). Quelle: `pipeline_summary.csv`.

Datensatz	Modell	FP32	PTQ	QAT	PTQ-Verlust	QAT-Erholung
CIFAR10	CNN	81.30	71.08	84.46	-10.22	+13.38
CIFAR10	ResNet-18	84.18	82.76	90.51	-1.42	+7.75
CIFAR10	ResNet-50	81.14	79.54	86.80	-1.60	+7.26
ImageNet100	CNN	43.04	25.06	51.18	-17.98	+26.12
ImageNet100	ResNet-18	80.06	71.30	82.52	-8.76	+11.22
ImageNet100	ResNet-50	80.02	64.06	83.22	-15.96	+19.16

Tabelle 2: Gewichts- vs. Aktivierungs-Dekomposition, CIFAR10 (Prozentpunkte gegenüber FP32; negativ = Verbesserung). Quelle: `activation_decomposition.csv`.

Modell	Stufe	Gewicht	Akt.	dominant
CNN	PTQ	10.24	-0.07	Gewicht
CNN	QAT	-3.22	3.11	Aktivierung
ResNet-18	PTQ	1.41	0.09	Gewicht
ResNet-18	QAT	-6.36	-5.48	Aktivierung
ResNet-50	PTQ	1.90	-0.23	Gewicht
ResNet-50	QAT	-5.50	-3.72	Aktivierung

bereits methodisch an, dass Datensatz und Auflösung die Quantisierungsempfindlichkeit mitbestimmen.

5.2 Phase 1: Gewichts- vs. Aktivierungs-Dekomposition

Tabelle 2 zerlegt den PTQ- und QAT-Genauigkeitsverlust in einen Gewichts- und einen Aktivierungsanteil (Quelle: `activation_decomposition.csv`; ausschließlich für CIFAR10 verfügbar). Unter PTQ dominiert in allen drei Architekturen der Gewichtsanteil den Genauigkeitsverlust, am stärksten beim `cnn` (10.24 gegenüber -0.07 Punkten), am moderatesten bei ResNet-50 (1.90 gegenüber -0.23 Punkten). Unter QAT kehrt sich das Bild um. Der isolierte Aktivierungspfad verliert relativ stärker, während der isolierte Gewichtspfad die FP32-Baseline übertrifft.

Da die Hessian-basierte Sensitivitätsanalyse (§4.3–4.4) nur Gewichtsschaden erfasst, ist der Anwendungsbereich durch diese Dekomposition abgesteckt. Unter PTQ, wo die Gewichte den Verlust dominieren, adressiert die Hessian-Analyse den Hauptmechanismus. Eine Übertragung auf die QAT-Sensitivität wäre dagegen fragwürdig, da dort die Aktivierungen relativ stärker zum Verlust beitragen. Dies ist ein Grund, warum die Hauptanalyse auf PTQ fokussiert wird (ImageNet100-Daten zu dieser Dekomposition

Tabelle 3: `conv1`-Trace bei Zufallsinitialisierung vs. trainiert (FP32, unfusioniert). Klassifikation laut `random_init_comparison.csv`.

Modell	Random ($\mu \pm \sigma$)	Trainiert	Klasse
CNN	0.0024 ± 0.0001	65.87	gelernt
ResNet-18	7.22 ± 1.19	17.35	gelernt
ResNet-50	74.4 ± 68.6	15.55	gelernt

liegen nicht vor, siehe §6.6).

Ergänzend zeigt die Aktivierungs-Ablationsanalyse (`activation_ablation.csv`), dass das sukzessive Zurücksetzen der Schichten mit dem höchsten Aktivierungs-Ausreißerfaktor (bis zu Faktor $25.1\times$ bei ResNet-50-layer4.1.conv3) auf FP32 nur geringe Genauigkeit zurückgewinnt (etwa +0.16 Punkte für ResNet-50/PTQ nach Ausschluss der fünf am stärksten ausreißenden Schichten). Aktivierungs-Ausreißer sind zwar messbar vorhanden (`outlier_factor` typischerweise $3\text{--}5\times$ bei Faltungsschichten, `activation_ranges.csv`), tragen aber unter PTQ nicht dominant zum Gesamtschaden bei.

5.3 Phase 2: Random-Init-Kontrolle

Tabelle 3 vergleicht die Trace der `conv1`-Schicht bei zufälliger Initialisierung (Mittelwert $\pm$ Standardabweichung über drei Init-Seeds) mit der Trace desselben Layouts nach Training (FP32, unfusionierte Basis; Quelle: `random_init_comparison.csv`, `random_init_summary.csv`).

In allen drei Modellen ist `conv1` als „gelernt“ klassifiziert. Beim `cnn` liegt die trainierte Trace um den Faktor ~ 27500 über der Zufalls-Trace, was die extremste der drei Verschiebungen darstellt. Bei ResNet-18 beträgt der Faktor ~ 2.4 , und bei ResNet-50 liegt die trainierte Trace unterhalb des mit sehr hoher Streuung behafteten Zufalls-Mittelwerts, wird aber dennoch als „gelernt“ eingestuft, da die Elevation relativ zum media-

nen Trace-Wert des jeweiligen Modells nach Training zunimmt (`elev_over_median_random= 0.10` vs. `elev_over_median_trained= 1.02`). Die `conv1`-Krümmungsspitze entsteht durch Training und ist nicht bereits eine Eigenschaft der Netzarchitektur bei Initialisierung. Für ResNet-50 zeigt der volle Trace-Profil-Vergleich (alle Schichten, nicht nur `conv1`) zusätzlich eine signifikante Spearman-Korrelation zwischen Random- und Trainiert-Profil ($\rho = 0.60$, $p < 0.001$) mit dem stärksten Ausreißer bei `layer4.1.conv2` (Elevation Random= $38.8\times$, Trainiert= $7.8\times$). Dies ist ein Hinweis darauf, dass bei ResNet-50 zusätzlich zur `conv1`-Dynamik auch architektonische, bereits bei Init vorhandene Effekte tiefer im Netz eine Rolle spielen, während beim ResNet-18-Profil keine signifikante Korrelation besteht ($\rho = 0.09$, $p = 0.70$).

Diese tiefenabhängige Struktur bei Zufallsinitialisierung ist qualitativ konsistent mit der Mean-Field-Vorhersage von Karakida, Akaho und Amari [12] (§2.4). Das Training kehrt dieses anfängliche Tiefenprofil für `conv1` um, indem es dessen Krümmung relativ zum Rest des Netzes anhebt.

5.4 Phase 3: Trace-Drift und Rekonziliation

Im Projektverlauf entstanden für dieselbe nominelle Größe, die PTQ-Trace von ResNet-50-`conv1`, unterschiedliche historische Werte, abhängig davon, welche nicht dokumentierte Konfiguration ein früherer Lauf verwendete, nämlich 13 161.8 in einem Lauf und 1030.9 in einem späteren. Die kanonische Neu-Messung (§4.2) ergibt 97.76 ± 6.27 (Mittelwert $\pm$ Standardabweichung über drei Proben-Seeds; `canonical_traces.csv`), was Verhältnissen von $134.6\times$ bzw. $10.5\times$ zu den historischen Werten entspricht.

Um diese Diskrepanzen zu erklären, wird ein Sweep über Basis (fusioniert/unfusioniert), Loss-Reduktion (Mittelwert/Summe), Proben-Anzahl, Datenquelle und Recheneinheit durchgeführt (`drift_diagnosis.csv`). Nur der FP32-Wert aus Lauf 1 lässt sich reproduzieren, und zwar durch die unfusionierte Basis mit einem Residuum von 2.8%. Keine getestete Konfiguration reproduziert die beiden historischen PTQ-Werte auch nur annähernd. Selbst die extremsten getesteten Varianten, also die Summenreduktion (14 101 für FP32, 7742 für PTQ), das kleinste Proben-Budget (`maxIter= 20`: 139.9 bzw. 82.5) oder ungeseedete Trainingsdaten mit Augmentation (286.0/184.4 bzw. 105.6/388.6), bleiben um mindestens eine Größenordnung von 13 161.8 entfernt.

Dies wird als dokumentierter, abgeschlossener Befund berichtet und nicht als offene Frage. Die historischen PTQ-Werte lassen sich mit den verfügbaren Konfigurationsknöpfen nicht rekonstruieren, und plausible Ursachen sind ein damals nicht protokollierter Checkpoint-, Kalibrierungs- oder Codepfad-Unterschied, der außerhalb des hier gesweepen Parameter-raums liegt. Die praktische Konsequenz ist die in §4.2 eingeführte. Ohne eine dokumentierte, fixierte Konfiguration sind Krümmungs-basierte Sensitivitätswerte zwischen Läufen, und potenziell zwischen Publikationen, nicht direkt vergleichbar.

5.5 Phase 3.5: Fusion vs. Quantisierung

Batch-Normalisierungs-Fusion allein erhöht die `conv1`-Trace bereits, bevor überhaupt quantisiert wird (Verhältnis fusioniert/unfusioniert, FP32, `canonical_traces.csv`): $11.7\times$ bei ResNet-50 (174.5 vs. 14.9), $10.7\times$ bei ResNet-18 (182.1 vs. 17.0), $7.3\times$ beim `cnn` (460.5 vs. 63.1). Tabelle 5 setzt dies in Bezug zur zusätzlichen Amplifikation durch PTQ selbst (`conv1_ptq_amplification = TrPTQ/TrFP32,fused`, sowie deren Rang unter allen Schichten des Modells; Quelle: `quant_induced_summary.csv`).

Nur beim `cnn` übersteigt die PTQ-Trace die bereits fusionierte FP32-Trace nennenswert (Faktor 1.15). Bei ResNet-18 und ResNet-50 liegt die PTQ-Trace unterhalb der fusionierten FP32-Trace (Faktoren 0.95 und 0.56), und in beiden Fällen liegt `conv1` in der PTQ-Amplifikations-Rangfolge nahe am unteren Ende (Rang 19 von 21 bzw. 51 von 54). Die dramatische `conv1`-Erhöhung, die in §5.3 beobachtet wurde, ist demnach primär ein Fusions-Effekt und nicht primär ein Quantisierungs-induzierter Effekt, mit der Ausnahme des `cnn`, wo PTQ selbst eine zusätzliche Verstärkung beiträgt. Die `conv1`-Krümmung, die dort als Prädiktor-Signal dient, entsteht überwiegend vor der eigentlichen PoT-Rundung.

5.6 Phase 4: Prädiktor-Test

Tabelle 6 zeigt für alle sechs Kombinationen (PTQ, loss-basierter Schaden), welchen Rang S_{raw} der Schicht mit dem tatsächlich größten Schaden zuweist, sowie die top- k^* -Überlappung aller drei Scores (Quelle: `weight_ablation_loss_damage_correlation.csv`, `normalized_ranks_loss.csv`).

S_{raw} erreicht eine Rang-1-Identifikation in 3 von 6 Kombinationen und wird in der top- k^* -Überlappung von keinem der beiden anderen Scores übertroffen. In beiden `cnn`-Zeilen liegt es gleichauf, in den

Tabelle 4: Rekonziations-Ledger für ResNet-50-conv1: historische Werte gegen die kanonische Neu-Messung. Quelle: `trace_reconciliation_ledger.csv`.

Größe	Alter Wert	Kanonisch	Verhältnis	Erklärung
conv1 FP32 (Lauf 1)	13.87	14.90	0.93×	unfusionierte Basis (Residuum 2.8 %)
conv1 FP32 (Lauf 2)	11.79	14.90	0.79×	ungeklärt
conv1 PTQ (Lauf 1)	13161.8	97.76	134.6×	ungeklärt
conv1 PTQ (Lauf 2)	1030.9	97.76	10.5×	ungeklärt
conv1 Elevation FP32	14.0	11.73	1.19×	ungeklärt

Tabelle 5: Fusion- und PTQ-Amplifikation für conv1. Rang = Position von conv1 in der Rangfolge aller Schichten nach PTQ-Amplifikation (1 = am stärksten amplifiziert). Quelle: `quant_induced_summary.csv`.

Modell	Fusion	PTQ-Amp.	Rang	Verdikt
CNN	7.3×	1.15	6/6	quant_induced
ResNet-18	10.7×	0.95	19/21	fp32_intrinsic
ResNet-50	11.7×	0.56	51/54	fp32_intrinsic

übrigen vier Kombinationen ist es strikt ueberlegen. S_{pert} erreicht in keiner Kombination Rang 1 und versenkt die wahre Top-Schicht regelmäßig weit hinten in der Rangfolge, insbesondere wenn diese eine Faltungsschicht mit kleinem Fan-in ist (conv1, layer1.1.conv1). S_{hawq} erreicht ebenfalls nie Rang 1 und übertrifft S_{raws} top- k^* -Überlappung in keiner Kombination. ResNet-50/CIFAR10 ist der Fall, in dem S_{raw} selbst am stärksten versagt (Rang 40 von 54), konsistent damit, dass der Schaden dort am diffusesten über alle 54 Schichten verteilt ist. Hier erreicht keiner der drei Scores mehr als ein Drittel der wahren top- k^* -Menge. Von den sechs S_{raw} -Korrelationen in der Tabelle schließt nur eine, ResNet-50/CIFAR10, das 95%-Bootstrap-CI von Null aus. Die übrigen fuenf sind positiven Wertes, aber bei dieser Stichprobengröße ($n = 6$ bis 54 Schichten) statistisch nicht gesichert.

conv1 ist in 4 von 6 Zeilen der Tabelle die wahre Top-Damage-Schicht, und alle drei Rang-1-Erfolge betreffen diese Faelle. Um zu prüfen, ob S_{raws} Signal an conv1 selbst haengt oder allgemeiner ist, wird die gesamte Auswertung nach Entfernung von conv1 aus der Schichtmenge jedes Modells wiederholt ($k^* = \lceil 0.1n \rceil$ auf der reduzierten Schichtzahl neu berechnet. Für das `cnn` ($n = 5$ nach Entfernung) ist ein Bootstrap über Schichten Rauschen, die den beiden `cnn`-Zeilen nur der Vollstaendigkeit halber berichtet werden und tragen keine Evidenz. Für die beiden ResNets hingegen ist der Rang der jeweils neuen Top-Damage-Schicht unter S_{raw} zusammengebrochen. ImageNet100/ResNet-18 fällt von Rang 1 (von 21) auf Rang 19 (von 20), ImageNet100/ResNet-50 von Rang 1 (von 54) auf Rang 50 (von 53). In beiden Fällen war conv1 selbst die wahre Top-Damage-

Schicht, und S_{raw} erkennt die neue Top-Schicht (`fc` bzw. `layer1.1.conv2`) nach deren Entfernung nicht mehr. Von den drei Rang-1-Erfolgen aus Tabelle 6 übersteht nur CIFAR10/ResNet-18 ($n = 20$) diese Kontrolle, weil dessen Top-Damage-Schicht (`layer1.1.conv1`) ohnehin nie conv1 war. Fuer die beiden ResNet-Fälle ist S_{raws} Rang-1-Trefferquote damit 1 von 2, nicht 3 von 3, und die beiden verlorenen Treffer sind jene, in denen conv1 die Antwort war. ResNet-50/CIFAR10, der einzige Fall mit von Null verschiedenem Konfidenzintervall in Tabelle 6, bleibt auch ohne conv1 signifikant ($\rho = 0.33$, CI [0.05, 0.57]).

Da die Hessian-Spur-Schätzung ein iterativer Hutchinson-Hessian-Vektor-Produkt-Verfahren ist (bis zu 100 Iterationen pro Schicht, §4.2), stellt sich die Frage, ob dieser Aufwand gegenüber einfacheren Kennzahlen gerechtfertigt ist. Drei günstige Baselines ohne jede Hessian-Berechnung werden auf demselben, in `trace_config.json` eingefrorenen Probe-Batch und mit derselben Verlustreduktion (`CrossEntropyLoss, reduction=mmean`) berechnet. Tabelle 8 stellt sie S_{raw} gegenueber, mit identischer Spearman-/Bootstrap-/Rang-1-/top- k^* -Auswertung (Quelle: `cheap_baselines_loss.csv`).

Sowohl die Gradientennorm als auch die empirische Fisher-Diagonale erzielen in allen sechs Kombinationen eine vergleichbare oder leicht bessere Rang-1- und Top- k^* -Trefferquote als S_{raw} – allerdings bei einem Bruchteil des Rechenaufwands. Die resultierenden Schicht-Rangfolgen sind dabei nahezu identisch mit denen von S_{raw} (Spearman- ρ zur Gradientennorm: 0,94–1,00; zur Fisher-Diagonale: 0,92–1,00).

Bei Kreuzentropie mit Softmax gilt nahe dem Mi-

Tabelle 6: Rang des wahren Top-Damage-Layers unter S_{raw} , dessen Spearman- ρ mit 95%-Bootstrap-CI, und top- k^* -Überlappung aller drei Scores, PTQ, loss-basiert. $k^* = 1, 3, 6$ für CNN/ResNet-18/ResNet-50. Fett = CI schließt Null aus. Quelle: `normalized_ranks_loss.csv`, `bootstrap_ci_spearman_loss.csv`.

Datensatz	Modell (n)	Top-Damage-Layer	S_{raw} -Rang	S_{raw} ρ [95%-CI]	Top- k^* -Überlappung		
					S_{raw}	S_{pert}	S_{hawq}
ImageNet100	CNN (6)	<code>conv1</code>	3	0.54 [-1.00, 0.80]	0/1	0/1	0/1
ImageNet100	ResNet-18 (21)	<code>conv1</code>	1	0.10 [-0.43, 0.54]	1/3	0/3	0/3
ImageNet100	ResNet-50 (54)	<code>conv1</code>	1	0.13 [-0.17, 0.41]	1/6	0/6	0/6
CIFAR10	CNN (6)	<code>conv1</code>	2	0.77 [0.00, 1.00]	0/1	0/1	0/1
CIFAR10	ResNet-18 (21)	<code>layer1.1.conv1</code>	1	0.42 [-0.01, 0.75]	1/3	0/3	0/3
CIFAR10	ResNet-50 (54)	<code>fc</code>	40	0.36 [0.10, 0.58]	2/6	1/6	1/6

Tabelle 7: Wie Tabelle 6, aber nach Entfernung von `conv1` aus der Schichtmenge jedes Modells; $k^* = \lceil 0.1n \rceil$ auf der reduzierten Schichtzahl neu berechnet. $\Delta\rho = \rho_{\text{ohne conv1}} - \rho_{\text{mit conv1}}$ (Tabelle 6). Für das CNN ($n = 5$) ist ein Bootstrap über Schichten Rauschen; die Zeile wird nur der Vollständigkeit halber angegeben und trägt keine Evidenz. Quelle: `conv1_excluded_control_loss.csv` (neu berechnet aus `weight_ablation_canonical_v2(_imagnet)`), `weight_ablation_loss_damage.csv`; Methodik `src/analysis/predictor_eval.py`, identisch zu Tabelle 6).

Datensatz	Modell (n , ohne <code>conv1</code>)	Top-Damage-Layer	S_{raw} -Rang	S_{raw} ρ [95%-CI]	Top- k^* -Überlappung			$\Delta\rho(S_{\text{raw}})$
					S_{raw}	S_{pert}	S_{hawq}	
ImageNet100	CNN [†] (5)	<code>conv2</code>	2	0.50 [-1.00, 1.00]	0/1	0/1	0/1	-0.04
ImageNet100	ResNet-18 (20)	<code>fc</code>	19	-0.04 [-0.54, 0.41]	0/2	0/2	0/2	-0.14
ImageNet100	ResNet-50 (53)	<code>layer1.1.conv2</code>	50	0.08 [-0.20, 0.37]	0/6	0/6	0/6	-0.05
CIFAR10	CNN [†] (5)	<code>conv2</code>	1	0.70 [-0.67, 1.00]	1/1	0/1	0/1	-0.07
CIFAR10	ResNet-18 (20)	<code>layer1.1.conv1</code>	1	0.36 [-0.09, 0.72]	1/2	0/2	0/2	-0.07
CIFAR10	ResNet-50 (53)	<code>fc</code>	39	0.33 [0.05, 0.57]	1/6	1/6	1/6	-0.03

[†] $n = 5$ nach Entfernung von `conv1`: das Bootstrap-CI ist hier nicht aussagekräftig, siehe Text.

nimum $H \approx J^\top J$, wobei J die Jacobi-Matrix der Modellausgabe bezüglich der Gewichte ist. Dieselbe Struktur geht auch in die Fisher-Information ein.

Beim Gradienten überrascht die Übereinstimmung zunächst, da dieser bei Konvergenz gegen null geht, die Hessian-Spur $\text{Tr}(H_l)$ jedoch nicht. Schreibt man den Schichtgradienten jedoch als $\nabla_{W_l} \mathcal{L} = J_l^\top r$ (mit Schicht-Jacobi J_l und Ausgangsresiduum r), wird die Verbindung deutlicher, denn bei fortschreitender Konvergenz schrumpft r im selben Rückwärtsdurchlauf für alle Schichten gleichermaßen. Das absolute Niveau des Gradienten sinkt zwar, seine relative Verteilung über die Schichten wird aber weiterhin durch J_l bestimmt, also genau durch den Term, der auch $\text{Tr}(H_l) \approx \text{Tr}(J_l^\top J_l)$ dominiert. Das relative Ranking bleibt dadurch stabil. Die naheliegende Sorge, der Gradient sei nahe dem Minimum zu klein oder rauschdominiert (§2.2), betrifft also nur die absoluten Werte, nicht aber die Schicht-Rangfolge.

Die reine Gewichtsnorm ist die einzige der drei Baselines, die als Prädiktor versagt. Sie korreliert instabil und teils stark negativ mit S_{raw} ($\rho = -0.83$ bis 0.42).

Abbildung 2 bündelt diese sechs Einzelfälle zu einer über die Architekturen normalisierten Zusammenfassung. Für eine Rang-Schwelle $t \in [0, 1]$ zeigt sie den Anteil der sechs PTQ-Kombinationen, in denen der jeweilige Score die wahre Top-Damage-Schicht innerhalb der obersten t -Schichten platziert. S_{raw} dominiert über den gesamten Schwellwertbereich, erreicht bereits bei $t = 0.05$ eine Trefferquote von 3/6 und bei $t = 0.5$ von 5/6, während S_{pert} selbst bei $t = 0.5$ noch keine einzige Kombination erfasst und S_{hawq} dazwischen liegt.

Abbildung 3 zeigt die vollstaendige Score-Wertschaden-Beziehung für alle sechs Kombinationen mit den zugehörigen Spearman-Korrelationen und Bootstrap-Konfidenzintervallen. Die Rangkorrelation ist insgesamt fragil. Von 18 Score×Kombination-Paaren sind 5 bei $p < 0.05$ signifikant, davon vier für S_{pert} bzw. S_{hawq} mit negativem Vorzeichen. S_{pert} korreliert in 5 von 6 Kombinationen negativ mit dem Schaden, entgegengesetzt zu S_{raw} s positivem Vorzeichen in denselben 5 Kombinationen.

Eine parameter-normalisierte Variante von S_{raw} (Division durch die Parameterzahl der Schicht)

Tabelle 8: S_{raw} vs. drei Baselines ohne Hessian-Berechnung (Gewichtsnorm, Gradientennorm, empirische Fisher-Diagonale), PTQ, loss-basiert, identische Auswertung wie Tabelle 6. Fett = Rang 1 bzw. CI schließt Null aus. Quelle: `cheap_baselines_loss.csv`.

Datensatz	Modell	$S_{\text{raw}} \rho$	$\ W\ \rho$	$\ \nabla\ \rho$	Fisher ρ
ImageNet100	CNN	0.54	-0.83	0.54	0.60
ImageNet100	ResNet-18	0.10	-0.21	0.09	0.12
ImageNet100	ResNet-50	0.14	-0.08	0.15	0.14
CIFAR10	CNN	0.77	-0.71	0.77	0.77
CIFAR10	ResNet-18	0.42	-0.59	0.47	0.43
CIFAR10	ResNet-50	0.36	0.20	0.37	0.38

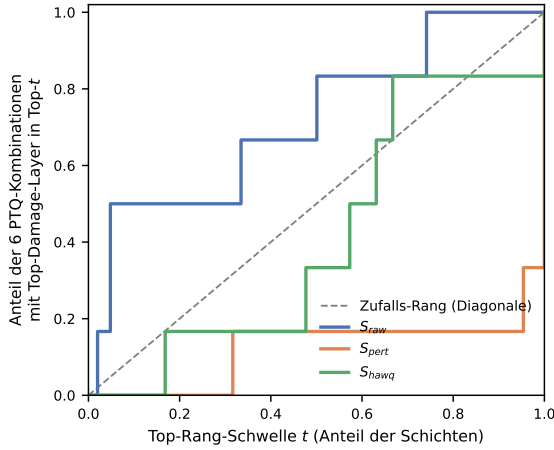


Abbildung 2: Kumulative Rang-Kurven: Anteil der 6 PTQ-Kombinationen, in denen die wahre Top-Damage-Schicht innerhalb der obersten t -Schichten (x-Achse) liegt, pro Score. Graue Diagonale = Zufalls-Rang-Erwartung. Erzeugt von `notebooks/16_full_ranking_curves.ipynb`.

verbessert die Korrelation in 5 von 6 Kombinationen, verschlechtert sie aber bei ResNet-50/CIFAR10 ($\rho = 0.36 \rightarrow 0.16$; Quelle: `raw_trace_per_param_comparison_loss.csv`). Da kein Normalisierungsschema universell ueberlegen ist, bleibt die unnormalisierte Form das Hauptresultat (§4.4).

Tabelle 9 zeigt die paarweise Spearman-Korrelation zwischen den drei Score-Werten selbst (unabhängig vom Schaden-Ziel; Quelle: `predictor_concordance.csv`). Bei ResNet-50 sind S_{pert} und S_{hawq} nahezu identisch ($\rho = 0.84$ CIFAR10, $\rho = 0.83$ ImageNet100), sodass der Drei-Wege-Vergleich dort effektiv zu einem Zwei-Wege-Vergleich wird (S_{raw} gegen ein fast identisches $S_{\text{pert}}/S_{\text{hawq}}$ -Paar). Dass S_{hawq} s top- k^* -ueberlappung bei ResNet-50 (Tabelle 6) S_{raw} nie uebertrifft, ist somit größtenteils S_{pert} s Muster, das durch das Produkt hindurchgereicht wird, und kein unabhängiges drittes Ergebnis.

Die vorstehende Analyse ist auf PTQ skaliert. Der direkte Vergleich mit der QAT-Krümmungsstruktur, der F1 beantwortet, folgt in §5.9.

5.7 Phase 5: Spike-Selection

Tabelle 10 vergleicht für CIFAR10 die Schicht mit der höchsten Krümmung im trainierten FP32-Modell („Trace-Spike“) mit der Schicht mit dem größten gemessenen Loss-Schaden („Damage-Spike“), Quelle `spike_selection.csv` (kanonische fusionierte Basis für die Trace-Spike-Bestimmung).

Unter der kanonischen fusionierten Basis stimmen Trace-Spike und Damage-Spike nur bei ResNet-18, also in 1 von 3 CIFAR10-Modellen, überein. Beim `cnn` liegt die Trace-Spitze bei `conv2`, während `conv1` am stärksten geschädigt wird. Bei ResNet-50 liegt die Trace-Spitze bei `conv1` (Fusions-getrieben, §5.5), während `fc` am stärksten geschädigt wird. Auf ImageNet100 (ebenfalls `spike_selection.csv`) fällt die Übereinstimmung hoeher aus. ResNet-50 und ResNet-18 stimmen beide überein (`conv1` ist dort sowohl Trace- als auch Damage-Spike), nur das `cnn` weicht ab (Trace-Spitze `conv3`, Damage-Spitze `conv1`), womit 2 von 3 ImageNet100-Modellen übereinstimmen.

Verwendet man statt der kanonischen fusionierten FP32-Trace die Trace-Elevation relativ zur Zufallsinitialisierung aus der Random-Init-Vergleichsanalyse (`random_init_summary.csv`, unfusionierte Basis, §5.3), so ergibt sich für CIFAR10 eine andere Trace-Spike-Schicht pro Modell (ResNet-50: `layer4.1.conv2`; ResNet-18: `layer3.0.conv2`. `Cnn`: `conv3`), und keines der drei Modelle stimmt mehr mit seinem Damage-Spike überein. Dieser Kontrast ist selbst ein Ergebnis. Ob die naive Heuristik „höchste Krümmung = größter Schaden“ zutrifft, hängt auch von der gewählten Trace-Konfiguration ab.

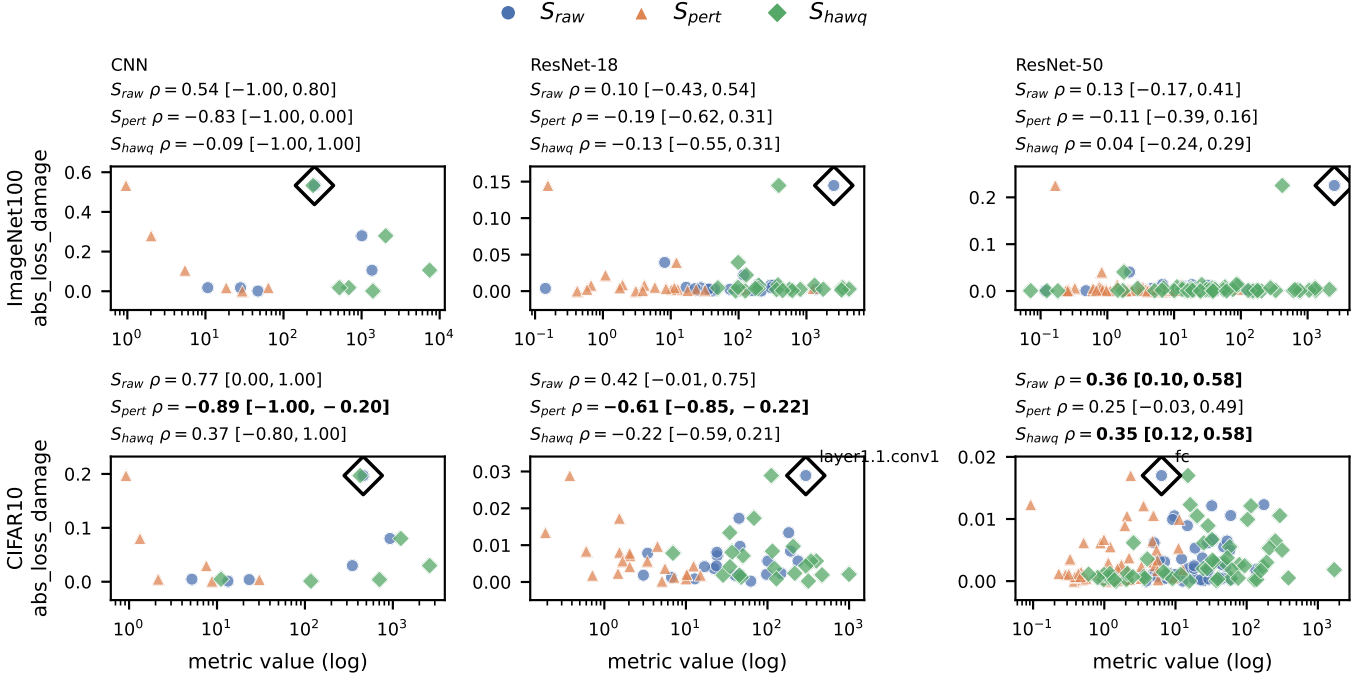


Abbildung 3: Score-Wert (log-Skala) vs. gemessener `abs_loss_damage`, PTQ; obere Zeile ImageNet100, untere Zeile CIFAR10, Spalten CNN/ResNet-18/ResNet-50. Offene Diamanten markieren die wahre Top-Damage-Schicht (beschriftet, wo nicht `conv1`). Jedes Panel-Inset zeigt Spearman- ρ mit 95%-Bootstrap-CI ($B = 2000$); fett = CI schließt Null aus. Erzeugt von `notebooks/06_main_facet_plot.ipynb`.

Tabelle 9: Paarweise Spearman- ρ zwischen den drei Score-Werten, PTQ. Quelle: `predictor_concordance.csv`.

Modell	Datensatz	$\rho(S_{raw}, S_{pert})$	$\rho(S_{raw}, S_{hawq})$	$\rho(S_{pert}, S_{hawq})$
CNN	ImageNet100	-0.71	0.60	0.09
CNN	CIFAR10	-0.54	0.71	0.03
ResNet-18	ImageNet100	-0.60	0.43	0.41
ResNet-18	CIFAR10	-0.55	0.45	0.43
ResNet-50	ImageNet100	0.40	0.78	0.83
ResNet-50	CIFAR10	0.48	0.86	0.84

5.8 Phase 6: Architektonische Ursachen-suche

Für die jeweilige Trace-Spike-Schicht jedes Modells (§5.7) wird versucht, die gemessene Pro-Parameter-Trace über die KFAC-Faktorisierung $\text{Tr}(F) \approx \text{Tr}(A) \cdot \text{Tr}(G)$ vorherzusagen (§2.4). Tabelle 11 zeigt, dass die Zerlegung in allen getesteten Fällen numerisch scheitert, da die vorhergesagte Pro-Parameter-Trace von der tatsächlich gemessenen um 3 bis über 8 Größenordnungen abweicht (Quelle: `spike_layer_kfac.csv`).

Die Abweichung ist für Faltungsschichten (`conv1`, `conv2`, `layer1.1.conv1`) größer als fuer die vollverbundene `fc`-Schicht von ResNet-50 (gut 3 Größenordnungen). Dies spricht nicht fuer eine grundsätzliche architektonische Schwäche der KFAC-Näherung, welche ursprünglich für einzelne dichte Schichten

entwickelt [20] und erst später auf Faltungsschichten erweitert [7] wurde, sondern für einen Skalierungsfehler in dieser Implementierung der Faktor-Konstruktion. `fan_in`-`fan_out` ist für jede Faltungsschicht um den selben Faktor `kh`-`kw` größer als `numel` (in Tabelle 12 bzw. `spike_layer_descriptors.csv` nachrechenbar, z.B. Faktor 9 für jede 3×3 -Faltung, Faktor 1 für die 1×1 -`fc`-Schicht), wodurch `predicted_per_param` für Faltungsschichten zu klein ausfällt und zwar um genau den Faktor, der den beobachteten `conv`/`fc`-Unterschied miterklärt. Hinzu kommt, dass $\text{Tr}(A)$ und $\text{Tr}(G)$ beide über Batch und räumliche Ausgabepositionen gemeinsam gemittelt werden, als wäre jede der L Ausgabepositionen ein unabhängiges Batch-Beispiel, während der tatsächliche Gewichtsgradient einer Faltungsschicht die Beiträge aller L Positionen aufsummiert statt

Tabelle 10: Trace-Spike (höchste FP32-Trace, kanonische fusionierte Basis) vs. Damage-Spike (größter PTQ-Loss-Schaden), CIFAR10.

Modell	Trace-Spike	Damage-Spike	Übereinst.
CNN	conv2	conv1	nein
ResNet-18	layer1.1.conv1	layer1.1.conv1	ja
ResNet-50	conv1	fc	nein

Tabelle 11: KFAC-Vorhersage vs. gemessene Pro-Parameter-Trace für die jeweilige Trace-Spike-Schicht, PTQ. Verhältnis = gemessen / vorhergesagt. Quelle: `spike_layer_kfac.csv`.

Modell	Schicht	Vorhergesagt	Gemessen	Verhältnis
CNN	conv1	3.6×10^{-9}	0.614	1.7×10^8
CNN	conv2	2.3×10^{-9}	0.069	3.0×10^7
ResNet-18	layer1.1.conv1	3.5×10^{-11}	0.0069	2.0×10^8
ResNet-50	conv1	3.7×10^{-9}	0.0566	1.5×10^7
ResNet-50	fc	1.3×10^{-7}	3.3×10^{-4}	2.5×10^3

Tabelle 12: Strukturelle Deskriptoren der Trace-Spike-Schichten. Quelle: `spike_layer_descriptors.csv`.

Modell	Schicht	Fan-in	Fan-out	#Param.
CNN	conv1	27	288	864
CNN	conv2	288	576	18432
ResNet-18	layer1.1.conv1	576	576	36864
ResNet-50	conv1	27	576	1728
ResNet-50	fc	2048	10	20480

mittelt. Diese fehlende Rückskalierung (bei $L \approx 576$ – 1024 in den betrachteten Schichten) ist der plausibelste Beitrag zur verbleibenden Lücke. Auch die aus der Faktorisierung abgeleiteten A - und G -Anteile (`spike_A_share`, `spike_G_share`) liegen wiederholt außerhalb des Bereichs $[0, 1]$ (etwa `conv1`: A -Anteil 16.6, G -Anteil -15.6).

Die in §2.4 skizzierte Hoffnung, Krümmungsspitzen über eine Eingabe-/Gradienten-Zerlegung architektonisch zu erklären, lässt sich mit der verwendeten Implementierung im PoT-Kontext nicht begründen. Die drei Hypothesen zur architektonischen Ursache konnten entsprechend nicht diskriminiert werden. Die Spalte `hypothesis_supported` ist fuer alle getesteten Schicht-Deskriptor-Paare „undetermined“, unter anderem weil der fuer eine Diskriminierung vorgesehene Auflösungs-kontrast zwischen CIFAR10 und ImageNet100 (`resolution_contrast_ratio`) für keine Kombination auswertbar war.

5.9 QAT-Ergebnisse und PTQ-vs-QAT-Vergleich

Dieser Abschnitt beantwortet die Frage, wie unterscheiden sich PTQ und QAT in der schichtweisen Verlust-Krümmung, und wie sich das auf die Score-vs-Schaden-Beziehung auswirkt?

Vergleicht man die kanonische, fusionierte Hessian-Spur pro Schicht zwischen PTQ- und QAT-Checkpoint derselben Kombination, korrelieren beide Trace-Profilе durchweg sehr hoch ($\rho = 0.95$ – 1.00 , $p < 0.001$ in jedem Fall). Die relative Krümmungsordnung der Schichten ändert sich also kaum. Ob nachträglich (PTQ) oder von Beginn an quantisierungsbewusst (QAT) trainiert wurde, spielt dafür fast keine Rolle. Das ist stimmt mit dem Befund aus §5.5 ueberein, dass die dominanten Krümmungsspitzen primär architektonisch (Fusion, Fan-in) und nicht durch den Trainingspfad bedingt sind.

Abbildung 4 zeigt das QAT-Gegenstück zu Abbildung 3 (Bootstrap-Rho und CI hier direkt aus den QAT-Per-Layer-Werten neu berechnet, da `bootstrap_ci_spearman_loss.csv` nur PTQ abdeckt; Methodik identisch zu §4.4, $B = 2000$, Seed 42). Von den 18 loss-basierten QAT-Score×Kombination-Paaren ist keines bei $p < 0.05$ signifikant (kleinster p -Wert 0.10, ResNet-18/CIFAR10, S_{pert}); das ist sogar schwächer als das eine genauigkeitsbasierte QAT-Ergebnis, das im urspruenglichen Berichtsentwurf notiert wurde (ResNet-18/ImageNet100, S_{pert} , $\rho = -0.47$, $p = 0.031$; Quelle: `weight_ablation_canonical_correlation_v2.csv`).

Unter QAT verbessert die isolierte Quantisierung

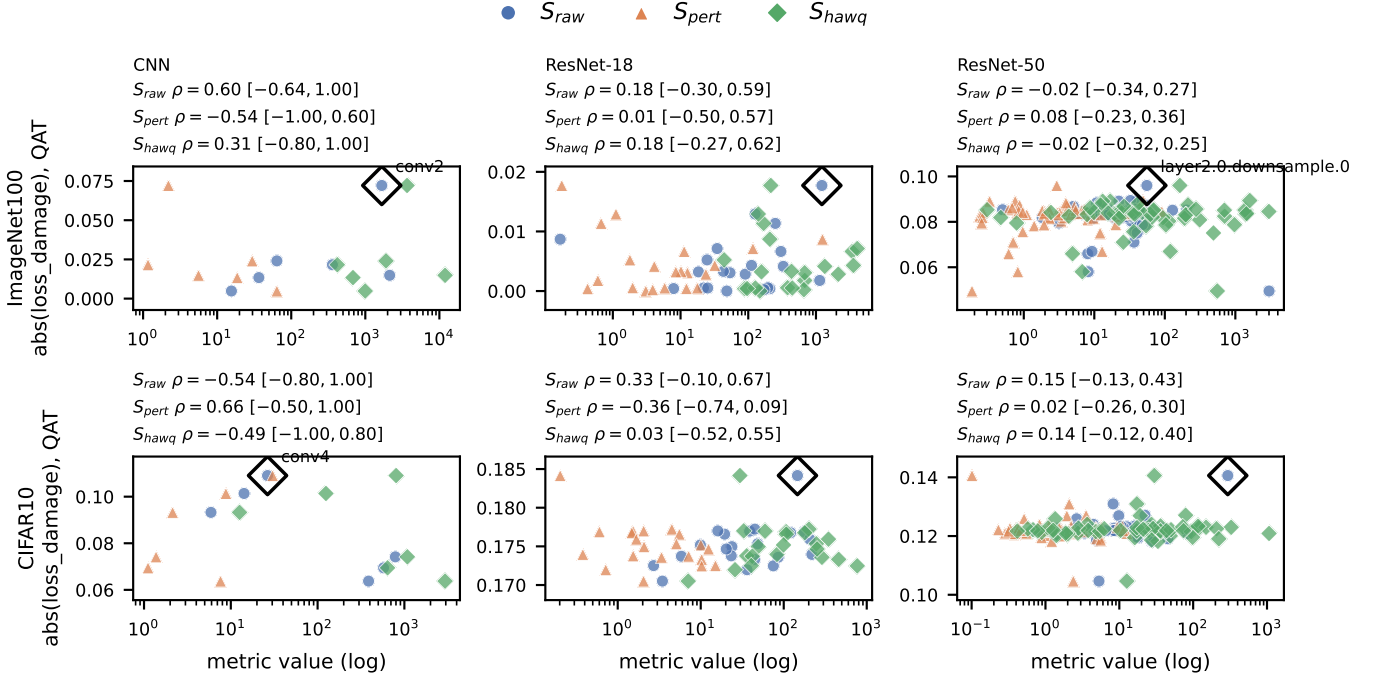


Abbildung 4: QAT-Gegenstück zu Abbildung 3: Score-Wert (log-Skala) vs. `abs_loss_damage`, QAT. Bootstrap-CI direkt aus den QAT-Werten berechnet ($B = 2000$, Seed 42). Erzeugt von `notebooks/14_qat_facet_plot.ipynb`.

einer Schicht den Loss häufig gegenüber der FP32-Baseline, was Vorzeichen und Größenordnung des Zielwerts gegenüber PTQ verändert. Die auf PTQ kalibrierte Score-Intuition, dass hohe Krümmung mit hohem Schaden einhergeht, überträgt sich damit nicht auf ein Regime, in dem „Schaden“ teils gar keiner ist. PTQ und QAT beeinflussen die relative Krümmungsstruktur eines Netzes kaum, sie formen aber, was diese Krümmung über den tatsächlichen Quantisierungseffekt aussagt.

5.10 Signed vs. Absolute Damage

Die Kernanalyse (§5.6) rankt Schichten nach `abs_loss_damage`, da ein reines Vorzeichen-Ranking Schäden beider Richtungen ungleich behandeln würde (§4.3). Abbildung 5 zeigt zur Einordnung dasselbe PTQ-Raster wie Abbildung 3, jedoch mit dem vorzeichenbehafteten `loss_damage` auf der y-Achse.

Unter PTQ ist der Vorzeichenwechsel selten, aber nicht vernachlässigbar. Von den 162 Schicht-Messungen über alle sechs Kombinationen sind 49 (30%) negativ, das heißt, die isolierte PTQ-Quantisierung dieser Schichten senkt den Loss geringfügig unter die FP32-Baseline (Quelle: `weight_ablation_loss_damage.csv`). Da diese negativen Werte betragsmäßig überwiegend klein sind, ändert sich die Identität der jeweils am stärksten geschädigten Schicht zwischen vorzeichenbehafteter und Betrags-Rangfolge in keiner der sechs Kombinationen. Der $|\cdot|$ -Betrag ist damit für das Ranking die richtige Wahl (§4.3), während die vorzeichenbehaftete Ansicht zusätzlich zeigt, dass „Schaden“ unter PTQ keine Einbahnstraße ist, ein Muster, das unter QAT (§5.9) noch ausgeprägter auftritt.

teter und Betrags-Rangfolge in keiner der sechs Kombinationen. Der $|\cdot|$ -Betrag ist damit für das Ranking die richtige Wahl (§4.3), während die vorzeichenbehaftete Ansicht zusätzlich zeigt, dass „Schaden“ unter PTQ keine Einbahnstraße ist, ein Muster, das unter QAT (§5.9) noch ausgeprägter auftritt.

5.11 Deployment-Ergebnisse

Tabelle 13 zeigt die gemessene mittlere Laufzeit-Beschleunigung (INT8 relativ zu FP32, über eine Reihe von Batch-Größen gemittelt) auf einer NVIDIA A100-GPU für zwei `torchao-0.17`-Konfigurationen. Die reine Gewichtsquantisierung (`int8_weight_only`) und die dynamische Aktivierungsquantisierung (`int8_dynamic_act`). Quelle ist ein projektinternes Diagnose-Protokoll (`int8_perf_diagnosis.txt`), das für jede der sechs Kombinationen und beide Quantisierungsstufen (PTQ/QAT-Checkpoint als Ausgangspunkt der Deployment-Konvertierung) einen vollständigen Batch-Größen-Sweep protokolliert.

Die dynamische Aktivierungsquantisierung ist in jeder Kombination langsamer als FP32 (Faktor 0.21–0.67, entsprechend 1.5–4.8× langsamer). Der Grund ist strukturell und in der Diagnose dokumentiert. `torchao 0.17s` Standard-Filterfunktion für `Int8DynamicActivationInt8WeightConfig` erfasst ausschließlich `nn.Linear`-Schichten

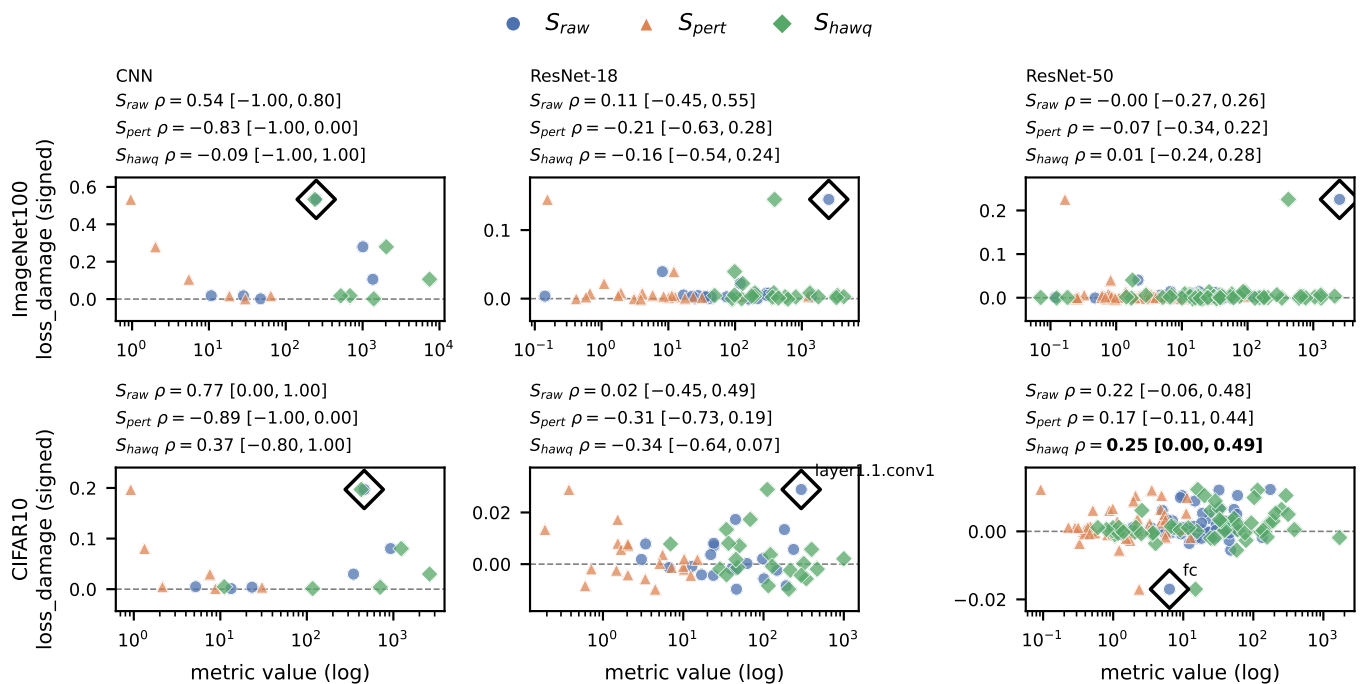


Abbildung 5: Wie Abbildung 3, jedoch mit vorzeichenbehaftetem `loss_damage` (PTQ) auf der y-Achse; gestrichelte Linie bei Null. Erzeugt von `notebooks/15_signed_damage_plot.ipynb`.

Tabelle 13: Mittlere INT8-vs-FP32-Laufzeit-Beschleunigung auf NVIDIA A100 (torchao 0.17), gemittelt über Batch-Größen 1–32. Werte > 1 = schneller als FP32. Quelle: `int8_perf_diagnosis.txt`.

Datensatz	Modell	wt.-only	dyn.-akt.
CIFAR10	CNN	1.05	0.21
CIFAR10	ResNet-18	1.19	0.56
CIFAR10	ResNet-50	1.12	0.65
ImageNet100	CNN	1.16	0.57
ImageNet100	ResNet-18	1.12	0.61
ImageNet100	ResNet-50	1.07	0.67

(`aten.linear/aten.mm`), nicht `aten.conv2d`; die einzige `Conv2d`-fähige Konfiguration, `IntxWeightOnlyConfig`, dequantisiert bei jedem Vorwärtsdurchlauf zurück auf FP32 und führt eine gewöhnliche FP32-Faltung aus. In dieser torchao-Version existiert schlicht kein optimierter INT8-Faltungskern. Der moderate Geschwindigkeitsgewinn von `int8_weight_only` (1.05 – $1.19\times$) stammt entsprechend nicht aus beschleunigter Faltungsberechnung, sondern daraus, dass diese Konfiguration die pro Aufruf anfallenden Aktivierungs-Quantisierungskosten der dynamischen Variante vermeidet, während sie dennoch den Gewichtsspeicher reduziert.

Das `fbgemm`-Backend von PyTorchs eager-mode-PTQ implementiert, im Gegensatz zum torchao-GPU-Pfad, echte quantisierte `Conv2d`- und `Linear`-Kernel für x86-Hardware mit AVX-512-VNNI (Zielsystem: AMD Ryzen 7 7700X, Zen 4,

16 Threads). Schichten werden in quantisierte `Conv2d`- und `Linear`-Module konvertiert und auf 50 Kalibrierungs-Batches kalibriert. Tabelle 14 zeigt den vollständigen Batch-Größen-Sweep für alle drei Modelle unter PTQ und QAT auf CIFAR10 (Quelle: `fbgemm_sweep.csv`) und Tabelle 15 die zugehörige Genauigkeit und Modellgrößen-Reduktion (Quelle: `fbgemm_summary.csv`). Anders als der torchao-GPU-Pfad realisiert `fbgemm` auf jeder getesteten Batch-Größe und in jeder Modell-/Stufe-Kombination einen Durchsatzgewinn gegenüber FP32 ($1.5\times$ bis $6.0\times$), bei einem Genauigkeitsverlust von unter einem Prozentpunkt, mit Ausnahme von ResNet-50 unter PTQ (-6.19 Punkte). ImageNet100 wurde mit diesem Backend nicht gemessen aufgrund von nicht verfügbaren Rechenressourcen.

`fbgemm` quantisiert auf echte INT8-Ganzzahlarithmetik und der gemessene Vorteil ist entsprechend real und einer INT8-Quantisierung

Tabelle 14: **fbgemm**-CPU-Backend (AMD Ryzen 7 7700X, AVX-512 VNNI, 16 Threads), echte quantisierte INT8-Conv2d/Linear-Kernel, CIFAR10 (mit diesem Backend nicht auf ImageNet100 gemessen). Beschleunigung INT8 relativ zu FP32 pro Batch-Größe. Quelle: **fbgemm_sweep.csv**.

Modell	Stufe	$bs=1$	$bs=8$	$bs=32$	$bs=64$	$bs=128$
CNN	PTQ	$1.59\times$	$2.29\times$	$2.62\times$	$3.16\times$	$4.81\times$
CNN	QAT	$1.50\times$	$2.28\times$	$2.79\times$	$3.25\times$	$4.39\times$
ResNet-18	PTQ	$6.02\times$	$3.07\times$	$3.09\times$	$3.47\times$	$4.34\times$
ResNet-18	QAT	$3.43\times$	$1.87\times$	$2.86\times$	$5.00\times$	$4.04\times$
ResNet-50	PTQ	$3.43\times$	$3.21\times$	$4.16\times$	$4.98\times$	$5.21\times$
ResNet-50	QAT	$3.75\times$	$3.25\times$	$4.12\times$	$5.05\times$	$5.17\times$

Tabelle 15: **fbgemm**-CPU: Genauigkeit und Modellgröße, INT8 vs. FP32, CIFAR10. Quelle: **fbgemm_summary.csv**.

Modell	Stufe	ΔAcc (in%)	Größe FP32→INT8	Reduktion
CNN	PTQ	-0.22	5.53 → 1.42 MB	$3.89\times$
CNN	QAT	-0.04	5.53 → 1.42 MB	$3.89\times$
ResNet-18	PTQ	-0.17	42.62 → 10.78 MB	$3.95\times$
ResNet-18	QAT	-0.09	42.62 → 10.78 MB	$3.95\times$
ResNet-50	PTQ	-6.19	89.85 → 23.00 MB	$3.91\times$
ResNet-50	QAT	-0.38	89.85 → 23.00 MB	$3.91\times$

zuzuschreiben. Er ist aber kein Test des PoT-Bitshift-Vorteils aus §2.1. **fbgemms** Kernel rechnen in Standard-INT8, nicht im PoT-Bitshift-Schema, und ein natives PoT-/Bitshift-Backend existiert fuer keine der beiden getesteten Hardware-Plattformen. Der Vorteil einer PoT-Bitshift-Implementierung gegenüber generischer INT8-Multiplikation bleibt damit nach diesem abgeschlossenen Sweep ungemessen.

Der PoT-Bitshift-Vorteil aus §2.1 übersetzt sich auf der getesteten GPU-Hardware nicht in einen gemessenen Laufzeitvorteil. Im Gegenteil, die nahelegendste torchao-Konfiguration ist langsamer als FP32. Der CPU-Pfad mit echten INT8-Kernen realisiert einen echten Durchsatzgewinn, aber für INT8 im Allgemeinen, nicht für PoTs Bitshift im Speziellen.

Ergänzend zum torchao-Pfad liegt für alle sechs Kombinationen eine unabhängige FPS-Messung in PyTorchs eager-mode-PTQ vor, ohne backend-optimierten Kernel (Quelle: **ptq_summary.csv**, **qat_summary.csv**). Tabelle 16 zeigt, dass auch hier PTQ und QAT in jeder Kombination langsamer sind als FP32 (Faktor 0.10–0.39), am auffälligsten bei ResNet-50 unter QAT (0.10–0.11 $\times$).

5.12 Accuracy-vs-Loss-Robustheits-Check

Um zu prüfen, ob die in §5.6 berichteten Befunde artefaktisch von der Wahl der Loss-basierten

Schaden-Definition (§4.3) abhängen, wird die identische Analyse mit genauigkeitsbasiertem Schaden wiederholt (**weight_ablation_canonical_v2.csv** und dessen ImageNet100-Variante, **weight_ablation_canonical_correlation_v2.csv**). Die qualitativen Kernaussagen bleiben unverändert. Die wahre Top-Damage-Schicht ist in allen 6 von 6 PTQ-Kombinationen identisch zwischen der Loss- und der Genauigkeits-basierten Definition (**conv1**, **conv1**, **layer1.1.conv1**, **conv1**, **fc**, **conv1** für die sechs Kombinationen aus Tabelle 6).

Was sich verschiebt, ist die Signifikanz-Zusammensetzung. Unter genauigkeitsbasiertem Schaden sind 3 von 18 Korrelationen bei $p < 0.05$ signifikant (**cnn**/CIFAR10 S_{raw} , ResNet-50/ImageNet100 S_{raw} , **cnn**/ImageNet100 S_{pert}), unter loss-basiertem Schaden 5 von 18 (§5.6), teils dieselben, teils andere Score $\times$ Kombination-Paare.

Abbildung 6 macht diese Übereinstimmung sichtbar, indem sie für alle 18 Score $\times$ Kombination-Paare die loss-basierte gegen die genauigkeitsbasierte Spearman- ρ aufträgt. Alle Punkte liegen nahe der Diagonalen $y = x$ und auf derselben Seite der Null-Linie wie ihr loss-basiertes Gegenstück.

Tabelle 16: FP32-, PTQ- und QAT-Durchsatz (Bilder/s) in PyTorchs eager-mode-PTQ, ohne Backend-Kernel. Quelle: `ptq_summary.csv`, `qat_summary.csv`.

Datensatz	Modell	FP32	PTQ	QAT	PTQ-Sp.	QAT-Sp.
CIFAR10	CNN	1027.6	397.1	390.4	0.39×	0.38×
CIFAR10	ResNet-18	306.2	79.5	37.7	0.26×	0.12×
CIFAR10	ResNet-50	133.6	27.6	13.8	0.21×	0.10×
ImageNet100	CNN	861.6	322.0	314.4	0.37×	0.36×
ImageNet100	ResNet-18	292.9	74.9	74.7	0.26×	0.25×
ImageNet100	ResNet-50	127.8	26.5	13.7	0.21×	0.11×

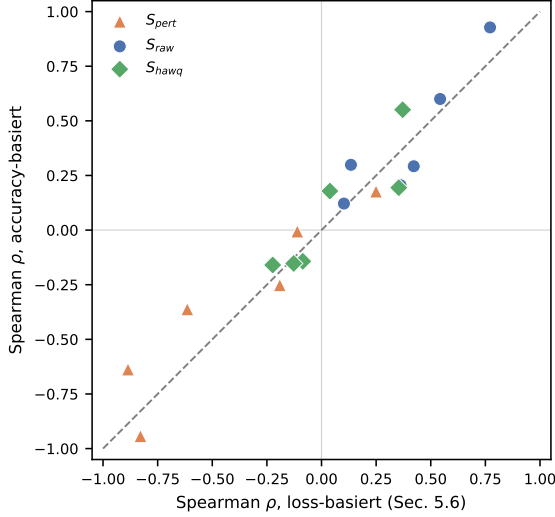


Abbildung 6: Spearman- ρ je der 18 Score×Kombination-Paare, loss-basiert (x-Achse, §5.6) gegen genauigkeitsbasiert (y-Achse, dieser Abschnitt). Graue Diagonale = $y = x$. Erzeugt von `notebooks/13_accuracy_loss_robustness.ipynb`.

6 Diskussion

6.1 Warum HAWQ-V2 unter PoT scheitert

Die Taylor-Motivation hinter dem HAWQ-V2-Produkt (Gleichung (1)) setzt voraus, dass die Störungsgröße $\|\Delta W_l\|^2$ über die Schichten hinweg vergleichbar skaliert, sodass die Krümmung $\text{Tr}(H_l)$ die eigentliche Rangfolge dominiert. Unter uniformer Quantisierung ist diese Annahme plausibel. Die Störungsgröße hängt dort im Wesentlichen von der gewählten Bit-Breite und dem Wertebereich einer Schicht ab, und beide Größen unterscheiden sich zwischen den Schichten eines Netzes oft nur moderat.

Unter PoT bricht diese Annahme zusammen. Wie in §2.1 erläutert, hängt die PoT-Störungsgröße primär davon ab, wie gut die Gewichtswerte einer Schicht zufällig mit dem logarithmischen PoT-Gitter übereinstimmen, was eine weitgehend von der

Krümmung dieser Schicht unabhängige Eigenschaft ist. `conv1` hat in den meisten Kombinationen die höchste Sensitivität (höchster S_{raw} -Rang, Tabelle 6), aber eine der kleinsten Störungsgrößen, sodass die beiden Faktoren des HAWQ-V2-Produkts hier entgegengesetztes Verhalten zeigen. Damit korreliert S_{pert} in 5 von 6 Kombinationen negativ mit dem gemessenen Schaden, entgegengesetzt zu S_{raw} s positiver Korrelation in denselben 5 Kombinationen (§5.6). Das Produkt S_{hawq} wird dadurch nach unten gezogen. Statt dass sich Krümmungs- und Störungssignal wie in der Taylor-Motivation vorgesehen sinnvoll ergänzen, heben sie sich teilweise auf.

Es ist kein allgemeines Scheitern Krümmungsbasierter Sensitivität, sondern ein Scheitern der Produktbildung genau unter PoT. Das Produkt funktioniert nur dann als sinnvoller Sensitivitäts-Score, wenn $\|\Delta W\|^2$ zumindest unkorreliert mit der Krümmungsbasierten Sensitivität einer Schicht ist. PoT verletzt dies strukturell, da die Gitter-Ausrichtung eine Eigenschaft der Gewichtsverteilung und unabhängig von der Krümmung ist, und für `conv1` sind beide direkt gegenläufig. Dies erklärt, warum eine etablierte, breit genutzte Sensitivitätsformulierung unter diesem Quantisierungsschema in die Irre führt.

6.2 Konfigurations-Sensitivität als methodisches Problem

Dass zwischen den historischen und den kanonischen Werten derselben nominellen Größe zwei Größenordnungen Abweichung liegen, ohne dass eine einzelne Konfigurationsentscheidung im gesweepen Parameterraum sie erklärt, zeigt, wie fragil ein einzelner, nicht dokumentierter Trace-Wert als Forschungsartefakt ist. Publierte Krümmungsbasierte Sensitivitätsanalysen, die ihre Konfiguration (Loss-Reduktion, BN-Fusionsstatus, Proben-Budget, Datensplit, Zufalls-Seeds) nicht dokumentieren, sind mit hoher Wahrscheinlichkeit schwer bis gar nicht reproduzierbar. Ursächlich ist nicht die Methode

selbst, sondern der Wertebereich, den unterschiedliche plausible Konfigurationsentscheidungen erzeugen können, in den eigenen Messungen mehrere Größenordnungen umfasst (`drift_diagnosis.csv`, §5.4).

Daraus lässt sich eine konkrete Empfehlung für zukünftige Arbeiten in diesem Bereich ableiten. Eine kanonische, versionierte Konfigurationsdatei (wie die vorliegende `trace_config.json`) sollte Standardpraxis sein, ebenso wie das Protokollieren, welche historischen Werte mit welcher Konfiguration reproduziert werden konnten und welche nicht, statt nur den zuletzt gemessenen Wert ohne diese Historie zu berichten.

6.3 Gewichts- vs. Aktivierungsregime

Die Dekomposition aus §5.2 zeigt, dass unter PTQ in allen drei getesteten Architekturen der Gewichtsanteil den Genauigkeitsverlust auf CIFAR10 dominiert. Da die Hessian-Analyse ausschließlich Gewichtsschaden erfasst, deckt sie damit, zumindest fuer PTQ auf CIFAR10, den tatsächlich dominierenden Verlustmechanismus ab. Unter QAT kehrt sich das Bild um. Dort dominiert der Aktivierungsanteil, und eine Übertragung der Hessian-basierten Gewichts-Sensitivitätsanalyse auf diesen Fall wäre nicht gerechtfertigt. Diese Beobachtung, konsistent mit der grundsatzlich unterschiedlichen Natur von Gewichts- und Aktivierungsschaden aus §2.5, ist der Kern der Antwort auf die dritte Forschungsfrage. Die Hessian-basierte Sensitivitätsanalyse ist nicht universell auf jedes Quantisierungsregime anwendbar und sollte vor Anwendbarkeit jedes Einsatzes geprüft werden, etwa durch eine Dekomposition wie in §5.2, statt vorausgesetzt zu werden.

6.4 PTQ vs. QAT Krümmungsstruktur

Die Ergebnisse aus §5.9 kombinieren zwei Befunde, die zunächst schwer zusammenpassen. QAT verändert die relative schichtweise Krümmungsstruktur kaum ($\rho = 0.95\text{--}1.00$ zwischen den PTQ- und QAT-Trace-Profilen über alle sechs Kombinationen). Die Beziehung zwischen dieser Krümmung und dem gemessenen Schaden bricht unter QAT dagegen fast vollständig zusammen (0 von 18 loss-basierten Korrelationen signifikant gegenüber 5 von 18 unter PTQ, §5.6). Unter QAT hat sich das Netz bereits an den Quantisierungsfehler angepasst (§2.1), sodass die isolierte Quantisierung einer Schicht den Loss oft nicht erhöht, sondern leicht senkt (§5.2). Die Krümmung bleibt ein stabiles Merkmal der Architektur; ihre Deutung als Schadens-Prädiktor setzt aber voraus,

dass eine Störung überhaupt schadet, was PTQ erfüllt und QAT durch sein Trainingsziel unterläuft. Für die Praxis heißt das, dass eine für PTQ kalibrierte Hessian-Sensitivitätsanalyse nicht ungeprüft auf QAT-Modelle übertragen werden sollte, auch wenn die Krümmungslandschaft nahezu identisch aussieht.

6.5 Deployment-Realität vs. theoretischer Vorteil

Der in §2.1 beschriebene Bitshift-Vorteil von PoT ist eine Eigenschaft der Arithmetik und nicht automatisch eine Eigenschaft jeder verfügbaren Software-/Hardware-Kombination. Die Deployment-Ergebnisse (§5.11) zeigen diesen Unterschied. Auf einer Standard-Rechenzentrums-GPU (A100) mit einem aktuellen, aber in seiner Faltungs-Kernel-Abdeckung eingeschränkten Quantisierungs-Framework (torchao 0.17) bringt die INT8-Quantisierung keinen Laufzeitvorteil und ist in der naheliegendsten Konfiguration langsamer als FP32. Auf einer CPU mit dedizierter INT8-Hardware-Unterstützung (AVX-512 VNNI) und einem Backend, das tatsächlich quantisierte Faltungs-Kernel implementiert (fbgemm), wird dagegen ein realer Durchsatzgewinn gemessen (1.5–6.0×, CIFAR10, Tabelle 14). Fbgemm beschleunigt generische Ganzzahl-Multiplikation, nicht den PoT Bitshift. Der theoretische Bitshift-Vorteil selbst bleibt in diesem Projekt auf keiner der beiden getesteten Plattformen gemessen, mangels eines nativen PoT-/Bitshift-Kernels.

Ob ein hardware-attraktives Schema real schneller ist, hängt entscheidend von der Reife der Inferenz-Software ab, nicht nur vom Schema selbst. Die Speicherreduktion (theoretisch 4× für Gewichte, §5.11) bleibt der arithmetisch verlässlichere Vorteil, unabhängig davon, ob ein optimierter Rechenkern existiert.

6.6 Limitationen

- **Ein Trainings-Seed pro Kombination:** Für keine der sechs Architektur×Datensatz-Kombinationen wurde über mehrere Trainings-Seeds gemittelt. Die Cross-Seed-Varianz der Top-Damage-Layer-Identifikation ist daher nicht charakterisiert; einzelne Schicht-Ränge könnten sich unter einem anderen Trainings-Seed verschieben. Als robust gilt das qualitative Muster, also die Top-Damage-Layer-Identität, der S_{raw} -vs- S_{hawq} -Vergleich und die $\text{top-}k^*$ -Überlappungen, nicht die Einzelrang-Stabilität.

- **Ungleiche Proben-Budgets zwischen Datensätzen:** CIFAR10 und ImageNet100 wurden mit unterschiedlichem Hutchinson-Proben-Budget vermessen, aus rechnerischer Notwendigkeit. Ein Teil der schwächeren Korrelation auf CIFAR10 (§5.6) könnte auf Messrauschen statt auf einen echten Datensatzeffekt zurückgehen.
- **Drei Architekturen, zwei Datensätze:** Die Generalisierbarkeit auf andere CNN-Familien (z.B. EfficientNet, MobileNet) oder auf nicht-Vision-Domänen ist ungetestet.
- **Nur PoT:** Die Kernaussage, dass das HAWQ-V2-Produkt durch PoT-Grid-Alignment-Effekte irreführt wird (§6.1), ist eine PoT Erklärung. Ob und wie stark sie sich auf uniforme oder gemischt-präzise Schemata überträgt, ist nicht getestet.
- **PTQ als primärer Fokus:** QAT-Ergebnisse (§5.9) werden berichtet, sind aber schwächer signifikant und methodisch komplizierter zu interpretieren, da isolierte QAT-Quantisierung den Loss teils verbessert statt verschlechtert.
- **KFAC-Zerlegung gescheitert:** Die architektonische Ursachensuche über KFAC liefert keine belastbare Erklärung; die drei aufgestellten Hypothesen bleiben unentschieden.
- **ImageNet100-Auflösungskontrast für Phase 6 nicht auswertbar:** Der für eine Diskriminierung der KFAC-Hypothesen vorgesehene Kontrast zwischen den Eingabeauflösungen von CIFAR10 und ImageNet100 (`resolution_contrast_ratio`) ist in den verfügbaren Daten nicht bestimmt.
- **fbgemm-CPU-Pfad nur auf CIFAR10 gemessen:** Der Durchsatz-/Genauigkeits-Sweep (§5.11, Tabelle 14) deckt alle drei Modelle und beide Quantisierungsstufen ab, aber nur auf CIFAR10. Für ImageNet100 liegt keine fbgemm-Messung vor. Zudem misst dieser Pfad INT8 im Allgemeinen, nicht PoTs Bitshift im Speziellen (§6.5). Der PoT Laufzeitvorteil bleibt auf beiden getesteten Plattformen ungemessen.

6.7 Ausblick

Mehrere Anschlussrichtungen ergeben sich direkt aus den hier dokumentierten Negativresultaten und offenen Fragen. Erstens könnte eine RMT-informierte

Zerlegung, die zwischen Bulk- und Ausreißer-Eigenwerten des Hessian-Spektrums unterscheidet, dort ansetzen, wo die KFAC-Faktorisierung gescheitert ist, möglicherweise mit einer gröberen, aber numerisch stabileren Zerlegung. Zweitens sollte die Erklärung dieses Projekts gegen uniforme und gemischt-präzise Quantisierungsschemata getestet werden, um zu prüfen, ob das HAWQ-V2-Scheitern tatsächlich PoT ist oder breiter auftritt. Drittens ist eine Ausweitung auf Transformer-Architekturen naheliegend, insbesondere da die einzige vorherige direkte Validierung der HAWQ-V2-Metrik [9] bereits im LLM-Setting stattfand (§3). Ein direkter CNN-vs-Transformer-Vergleich unter identischer Methodik wäre aufschlussreich. Viertens sollte eine Multi-Seed-Validierung die in §6.6 genannte Seed-Varianz-Lücke schließen.

7 Fazit

Dieses Projekt hat untersucht, ob Hessian-basierte Krümmungsmetriken vorhersagen können, welche Schichten eines CNN unter der Power-of-Two-Quantisierung am stärksten geschädigt werden, und liefert auf alle drei Forschungsfragen eine klare, wenn auch nuancierte Antwort.

Die Random-Init-Kontrolle (§5.3) zeigt, dass die für die Analyse `conv1`-Krümmungsspitze in allen drei Architekturen durch Training entsteht und nicht bereits bei Initialisierung vorhanden ist. Die Fusions-Analyse (§5.5) zeigt weiter, dass diese Spitze überwiegend ein Effekt der Batch-Normalisierungsfusion ist und nicht der PoT-Quantisierung selbst, mit Ausnahme des `cnn`, wo PTQ zusätzlich moderat verstärkt. Die relative Krümmungsstruktur selbst bleibt zwischen PTQ- und QAT-Checkpoints nahezu unverändert ($\rho = 0.95\text{--}1.00$ über alle sechs Kombinationen), während die Beziehung dieser Krümmung zum tatsächlichen Schaden unter QAT fast vollständig zusammenbricht (0 von 18 signifikant, gegenüber 5 von 18 unter PTQ). Die isolierte Schichtquantisierung verbessert unter QAT häufig den Loss gegenüber der FP32-Baseline, statt ihn zu verschlechtern (§5.2), sodass dieselbe Krümmungslandschaft ein anderes Zielsignal vorherzusagen versucht (§6.4).

Kein einzelner Score erweist sich als universell zuverlässig, um die am stärksten geschädigte Schicht zu identifizieren, und selbst die rohe Krümmung (S_{raw}) hält bei näherer Prüfung nur ein eng begrenztes Signal. Sie identifiziert zwar die wahre Top-Damage-Schicht in 3 von 6 Kombinationen (Rang 1), doch schließen 5 von 6 Bootstrap-Konfidenzintervallen

Null ein, und im einzigen signifikanten Fall (ResNet-50/CIFAR10) liegt dieselbe Top-Schicht auf Rang 40 von 54 (Tabelle 6). Eine Kontrollrechnung ohne `conv1` (Tabelle 7) zeigt, dass zwei der drei Rang-1-Erfolge direkt an `conv1` als wahre Top-Damage-Schicht gebunden sind, wobei ihr Rang nach dessen Entfernung auf 19 von 20 bzw. 50 von 53 faellt und dass `conv1`s Krümmungsspitze selbst überwiegend ein Fusions-/Fan-in-Artefakt ist (§5.5). Nur der ResNet-18/CIFAR10-Erfolg übersteht die Kontrolle, weil seine Top-Damage-Schicht ohnehin nie `conv1` war. Ein Vergleich mit zwei Baselines ohne jede Hessian-Berechnung zeigt zudem, dass beide in allen sechs Kombinationen dieselbe oder eine leicht bessere Rang- und top- k^* -Trefferquote erreichen wie S_{raw} , bei einem Bruchteil des Rechenaufwands. Das HAWQ-V2-Produkt scheitert zusätzlich daran, dass die PoT-Stoerungsgroesse mit der wahren Sensitivität anti-korreliert, statt neutral oder positiv zu sein (§6.1), was ein PoT Mechanismus und keine allgemeine Schwäche Krümmungs-basierter Sensitivität ist. Welche Schicht konkret am stärksten geschädigt wird, hängt zudem stark von Architektur und Datensatz ab (`conv1` auf ImageNet100 in allen drei Architekturen, aber uneinheitlich auf CIFAR10, Tabelle 6).

Unter PTQ auf CIFAR10 dominiert in allen drei Architekturen der Gewichtsschaden, unter QAT dominiert der Aktivierungsschaden (§5.2). Die Hessian-basierte Sensitivitätsanalyse, die ausschließlich Gewichtsschaden erfasst, ist damit nicht universell anwendbar, sondern regimeabhängig, und ihre Anwendbarkeit sollte vor jedem Einsatz geprüft werden.

Literatur

- [1] Yoshua Bengio, Nicholas Léonard und Aaron Courville. „Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation“. In: *arXiv preprint arXiv:1308.3432* (2013). arXiv: 1308.3432.
- [2] *CIFAR-10 and CIFAR-100 Datasets*. 26. Feb. 2026. URL: <https://www.cs.toronto.edu/~kriz/cifar.html> (besucht am 26.02.2026).
- [3] Zhen Dong u. a. „HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. arXiv: 1911.03852.
- [4] Zhen Dong u. a. „HAWQ: Hessian AWAre Quantization of Neural Networks with Mixed-Precision“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019. arXiv: 1905.03696.
- [5] Mahmoud Elgenedy. *Power-of-Two (PoT) Weights in Large Language Models (LLMs)*. 2025. arXiv: 2506.00315.
- [6] Amir Gholami u. a. *A Survey of Quantization Methods for Efficient Neural Network Inference*. 21. Juni 2021. DOI: 10.48550/arXiv.2103.13630. arXiv: 2103.13630 [cs]. URL: <http://arxiv.org/abs/2103.13630> (besucht am 09.11.2025). Vorveröffentlichung.
- [7] Roger Grosse und James Martens. „A Kronecker-factored Approximate Fisher Matrix for Convolution Layers“. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016. arXiv: 1602.01407.
- [8] Kaiming He u. a. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385.
- [9] Joshua Hill. *Saturation Makes Quantization Error Additive: A Coverage Model with a Certificate*. 2026. arXiv: 2607.12266 [cs.LG]. URL: <https://arxiv.org/abs/2607.12266>.
- [10] *ImageNet100*. URL: <https://www.kaggle.com/datasets/ambityga/imagenet100> (besucht am 11.09.2026).
- [11] Sergey Ioffe und Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv: 1502.03167.
- [12] Ryo Karakida, Shotaro Akaho und Shun-ichi Amari. „Universal Statistics of Fisher Information in Deep Neural Networks: Mean Field Approach“. In: *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2019. arXiv: 1806.01316.
- [13] Daya Khudia u. a. *FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference*. 2021. arXiv: 2101.05615.
- [14] Young Kim u. a. „Non-zero Grid for Accurate 2-Bit Additive Power-Of-Two CNN Quantization“. In: *IEEE Access* (2023).
- [15] Raghuraman Krishnamoorthi. *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper*. 21. Juni 2018. DOI: 10.48550/arXiv.1806.08342. arXiv: 1806.08342 [cs]. URL: <http://arxiv.org/abs/1806.08342>.

- 08342 (besucht am 26.02.2026). Vorveröffentlichung.
- [16] Edward H. Lee u. a. „LogNet: Energy-efficient Neural Networks Using Logarithmic Computation“. In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, S. 5900–5904.
 - [17] Xinlin Li u. a. „DenseShift: Towards Accurate and Efficient Low-Bit Power-of-Two Quantization“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, S. 16964–16974.
 - [18] Yuhang Li u. a. „BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction“. In: *International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2102.05426.
 - [19] Ji Lin u. a. „AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration“. In: *Proceedings of Machine Learning and Systems (MLSys)*. Best Paper Award. 2024. arXiv: 2306.00978.
 - [20] James Martens und Roger Grosse. „Optimizing Neural Networks with Kronecker-factored Approximate Curvature“. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015. arXiv: 1503.05671.
 - [21] Daisuke Miyashita, Edward H. Lee und Boris Murmann. „Convolutional Neural Networks using Logarithmic Data Representation“. In: *arXiv preprint arXiv:1603.01025* (2016). arXiv: 1603.01025.
 - [22] Shahmustafa Mujawar, Divya Kiran und Hariharan Ramasangu. „An efficient CNN architecture for image classification on FPGA accelerator“. In: *2018 Second International Conference on Advances in Electronics, Computers and Communications (ICAEECC)*. IEEE. 2018, S. 1–4.
 - [23] Markus Nagel u. a. *A White Paper on Neural Network Quantization*. 15. Juni 2021. DOI: 10.48550/arXiv.2106.08295. arXiv: 2106.08295 [cs]. URL: <http://arxiv.org/abs/2106.08295> (besucht am 06.01.2026). Vorveröffentlichung.
 - [24] Dominika Przewłocka-Rus u. a. „Power-of-Two Quantization for Low Bitwidth and Hardware Compliant Neural Networks“. In: *tinyML Research Symposium*. 2022. arXiv: 2203.05025.
 - [25] Levent Sagun, Léon Bottou und Yann LeCun. „Singularity of the Hessian in Deep Learning“. In: *arXiv preprint arXiv:1611.07476* (2016). arXiv: 1611.07476.
 - [26] Levent Sagun u. a. „Empirical Analysis of the Hessian of Over-Parametrized Neural Networks“. In: *arXiv preprint arXiv:1706.04454* (2017). ICLR 2018 Workshop. arXiv: 1706.04454.
 - [27] Yonglong Tian, Dilip Krishnan und Phillip Isola. *Contrastive Multiview Coding*. 2020. arXiv: 1906.05849 [cs.CV]. URL: <https://arxiv.org/abs/1906.05849>.
 - [28] Guangxuan Xiao u. a. „SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models“. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023. arXiv: 2211.10438.
 - [29] Zhewei Yao u. a. „PyHessian: Neural Networks Through the Lens of the Hessian“. In: *2020 IEEE International Conference on Big Data (Big Data)*. IEEE, 2020, S. 581–590. arXiv: 1912.07145.